

Politecnico di Torino

Master's Degree in Mechatronics Engineering

Academic Year 2025/2026

Graduation Session December 2026

Visual Autonomous Navigation for Horticulture

Supervisors:
Name Surname
Name Surname

Candidate:
Emanuele Pruiti

Abstract

Autonomous navigation for mobile agricultural robots in horticultural environments faces severe operational challenges under Global Navigation Satellite Systems (GNSS). Thick plant canopies, multi-path reflections, and greenhouse structures degrade or eliminate satellite reception, while Real-Time Kinematic (RTK) corrections incur high infrastructure costs. High-resolution 3D LiDAR sensors offer direct geometric measurements, but unit costs remain prohibitive for scalable commercial fleets, and spatial point clouds lack the chromatic and textural cues required to distinguish drivable soil from dense crop foliage.

This thesis presents a real-time, deterministic vision-based navigation system for row-following horticulture based on monocular RGB imagery. The architecture decouples offline model-assisted dataset curation from low-latency onboard execution. To minimize manual annotation overhead, the Segment Anything Model (SAM 3) generates initial binary corridor segmentations from prompt-guided field captures. For real-time inference on an embedded NVIDIA Jetson Orin Nano platform, a lightweight MobileNetV3 + LR-ASPP semantic segmentation network is deployed via ONNX Runtime with FP16 precision, executing at 20 Hz within a 15 ms inference budget.

From the predicted drivable corridor mask, navigation setpoints are extracted using a decoupled dual-edge geometric tracking algorithm. The region of interest is partitioned into horizontal scanline strips from bottom to top. Left and right physical crop boundaries are tracked independently using spatial continuity gates and fitted using Iteratively Reweighted Least Squares (IRLS) with a Cauchy M-estimator. The central navigation line is computed analytically from the converged boundary parameters.

The proposed algorithm is benchmarked against three established agricultural vision baselines: a global column-projection extractor (SegMin column histogram), a strip-centroid regression line extractor, and a contour-point line extractor fitted via Random Sample Consensus (RANSAC). Rather than claiming unconditional superiority, the thesis evaluates the structural trade-offs of each paradigm. The proposed dual-edge extractor eliminates the stochastic frame-to-frame angular jitter characteristic of RANSAC ($\sigma_\omega = 0.00^\circ$ versus non-zero variance) and prevents

unilateral leaf intrusions from biasing the corridor axis, a primary failure mode of centroid-based fitting. Conversely, experimental failure analysis formalizes key structural limitations of the proposed method: vulnerability to near-field seed occlusion, sensitivity of image-space jump gates to abrupt vehicle pitch and roll, inability of the straight-line model to track curved crop geometries or headland turnaround maneuvers, and boundary loss during extended unilateral crop gaps.

The perception and geometric extraction stages are validated offline across recorded horticultural datasets and benchmarked on the embedded NVIDIA Jetson Orin Nano, confirming real-time execution at 20 Hz with sub-15 ms latency without requiring secondary neural networks, stereo disparity maps, or LiDAR sensors. Finally, the thesis establishes the integrated control architecture and closed-loop field evaluation protocol on the Demetra four-wheel-steering (4WS) agricultural rover, using an auxiliary laser sensor to monitor physical corridor centering.

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Table of Contents

List of Figures VIII

1 Introduction 1
- 1.1 Context and motivation . . . 1
- 1.2 Objectives of the thesis . . . 2
- 1.3 Organization of the work . . . 3

2 Background and Problem Formulation 4
- 2.1 Agricultural autonomous navigation . . . 4
  - 2.1.1 Navigation tasks and requirements . . . 4
  - 2.1.2 Environmental challenges . . . 4
- 2.2 Vision-based local guidance . . . 5
  - 2.2.1 Sensing modalities . . . 5
  - 2.2.2 Image-space reference extraction . . . 5
- 2.3 The segmentation-to-control pipeline . . . 7
  - 2.3.1 Perception stage . . . 7
  - 2.3.2 Geometric extraction stage . . . 7
  - 2.3.3 Control integration . . . 8

3 State of the Art 9
- 3.1 Classical image-processing methods . . . 9
- 3.2 Learning-based segmentation for agricultural navigation . . . 9
  - 3.2.1 Semantic segmentation approaches . . . 9
  - 3.2.2 Training data and generalization . . . 10
- 3.3 Semantic segmentation architectures . . . 10
  - 3.3.1 FCN, DeepLabV3, and mobile architectures . . . 10
  - 3.3.2 Transfer learning and pretraining . . . 10
- 3.4 Dataset preparation and model-assisted annotation . . . 11
  - 3.4.1 Manual annotation challenges . . . 11
  - 3.4.2 SAM and foundation models for segmentation . . . 11
- 3.5 Geometric extraction methods . . . 12

3.5.1 Line fitting and crop-edge detection . . . 12
3.5.2 Column-wise histogram methods . . . 12
3.5.3 Horizontal strips and adaptive multi-ROI methods . . . 12
3.5.4 Triangle Scan Method . . . 13
3.5.5 Bilateral boundary tracking versus centroid reduction . . . 13
3.5.6 Comparison of geometric representations . . . 15
3.6 System integration and experimental validation . . . 16
3.6.1 Perception and control integration . . . 16
3.6.2 Scope of reported evaluations . . . 16
3.7 Positioning of this thesis . . . 16

4 Deep Learning Architectures for Semantic Segmentation 18
4.1 Problem formulation and encoder-decoder principles . . . 19
4.2 Fully Convolutional Networks (FCN) . . . 20
4.2.1 Convolutionalization of classification networks . . . 21
4.2.2 Skip architecture and upsampling . . . 21
4.3 DeepLabV3 and atrous convolutions . . . 22
4.3.1 Atrous convolution formulation . . . 23
4.3.2 Atrous Spatial Pyramid Pooling (ASPP) . . . 23
4.4 MobileNetV3 and the Lite Reduced ASPP (LR-ASPP) head . . . 24
4.4.1 Depthwise separable convolutions . . . 25
4.4.2 Inverted residual blocks with linear bottlenecks . . . 25
4.4.3 Squeeze-and-Excitation (SE) channel attention . . . 26
4.4.4 Efficient non-linearities: hard-swish and hard-sigmoid . . . 27
4.4.5 The Lite Reduced ASPP (LR-ASPP) architecture . . . 27
4.5 Segment Anything Model 3 (SAM 3) for dataset annotation . . . 28
4.5.1 Architecture overview . . . 30
4.5.2 Operational distinction from onboard models . . . 31
4.6 Summary of architectural characteristics . . . 31

5 Dataset Preparation and Comparative Model Training 33
5.1 Image acquisition and class definition . . . 34
5.1.1 Image sources and acquisition conditions . . . 34
5.1.2 Semantic classes and labeling targets . . . 34
5.2 Dataset preparation . . . 35
5.2.1 Lettuce dataset with manual annotation . . . 35
5.2.2 Automated zero-shot annotation via SAM 3 . . . 35
5.2.3 Tomato and cucumber images . . . 36
5.2.4 Dataset partitioning . . . 37
5.3 Architecture comparison: FCN, DeepLabV3, and LR-ASPP . . . 37
5.3.1 Model selection and pretrained weights . . . 38

5.3.2 Training configuration and loss formulation 38
5.3.3 Evaluation metrics and results 40
5.3.4 Selection of LR-ASPP 42
5.4 Supervision strategy comparison 43
5.4.1 Training configurations and evaluation targets 43
5.4.2 Benchmark results across supervision strategies 44
5.4.3 Path segmentation with autolabel supervision 47
5.4.4 Network deployed on the robot 49

6 Geometric Extraction of the Local Navigation Reference 50
6.1 Input representation and edge-line models 50
6.2 Region of interest and mask preparation 51
6.3 Horizontal strips and candidate intervals 52
6.3.1 Strip construction and column projection 52
6.3.2 Local support around interval boundaries 52
6.4 Initialization and bottom-up association 54
6.4.1 Edge-support seed selection and gap-limited association . . . 54
6.5 Separate reweighted fitting of the corridor edges 55
6.5.1 Excluding image-boundary observations 55
6.5.2 Residual-based reweighting 55
6.6 Quality indicators, validation, and image-space output 57
6.6.1 Method 4: geometric checks and a composite score 57
6.6.2 Interpretation of status and confidence 58
6.7 Default configuration 58
6.8 Effect of strip height and number 60
6.9 Baseline algorithms for comparative evaluation 61
6.9.1 Center-point strip tracking baseline 61
6.9.2 Column-histogram baseline 62
6.9.3 RANSAC-based contour baseline 62
6.10 Comparative trade-offs and structural limitations 63
6.10.1 Structural limitations of Method 4 63

7 Robot Platform and Integration 66
7.1 Robot hardware and sensors 66
7.1.1 Robot platform 66
7.1.2 Camera configuration 67
7.1.3 Computing platform 69
7.2 Software architecture 69
7.2.1 ROS integration 69
7.2.2 Perception pipeline implementation 74
7.2.3 Control interface 74

7.3 Deployment configuration . . . 75
7.3.1 Model export to ONNX . . . 75
7.3.2 Real-time inference . . . 75
7.3.3 Timing and latency . . . 76
7.3.4 Software-in-the-Loop pre-validation . . . 77

8 Experimental Validation 79
8.1 Offline evaluation protocol . . . 79
8.1.1 Segmentation metrics . . . 79
8.1.2 Geometric reference accuracy . . . 80
8.2 Field validation . . . 83
8.2.1 Ground-truth reference: laser sensor . . . 83
8.2.2 Field evaluation metrics . . . 84
8.2.3 Test conditions . . . 86
8.2.4 Closed-loop navigation results . . . 86
8.2.5 Qualitative observations . . . 87
8.3 Discussion . . . 87
8.3.1 Segmentation performance . . . 87
8.3.2 Geometric extraction trade-offs . . . 87
8.3.3 Limitations . . . 88

9 Conclusions and Future Work 90
9.1 Main contributions . . . 90
9.2 Conclusions on segmentation and annotation . . . 91
9.3 Conclusions on geometric extraction . . . 91
9.4 Limitations . . . 92
9.5 Future work . . . 94
9.5.1 Dataset and annotation . . . 94
9.5.2 Segmentation and model deployment . . . 94
9.5.3 Geometric extraction . . . 94
9.5.4 System integration and field validation . . . 95
9.6 Closing remarks . . . 95

Bibliography 96

List of Figures

2.1 Agricultural inter-row environment, vehicle frames, and geometric error definition 6
2.2 Complete vision-based local guidance pipeline from camera stream to actuator commands 7

4.1 Fully Convolutional Network (FCN) architecture with progressive skip connections combining deep semantic features with shallow appearance representations (adapted from Long et al. [9]). 22
4.2 DeepLabV3 architecture featuring the Atrous Spatial Pyramid Pooling (ASPP) module with parallel atrous convolutions and image-level pooling (adapted from Chen et al. [10]). 24
4.3 MobileNetV3 and Lite Reduced ASPP (LR-ASPP) architecture: (a) inverted residual block combining depthwise separable convolution, linear bottleneck, and Squeeze-and-Excitation channel attention; (b) segmentation head combining stride-16 features modulated by global context gating with stride-8 fine features (adapted from Howard et al. [11]). 29
4.4 Original Segment Anything Model (SAM) architecture with ViT image encoder, prompt encoder, and mask decoder (adapted from Kirillov et al. [12]). SAM 3 extends this foundation with a dual detector-tracker architecture and open-vocabulary concept segmentation. 31

5.1 Sample field images across crops and corresponding training masks 34
5.2 SAM 3 zero-shot autolabeling and mask composition workflow . . . 36
5.3 Training and validation loss across epochs for FCN, DeepLabV3, and LR-ASPP 40

6.1 Horizontal strip decomposition and column-wise projection for path interval detection 53

6.2 Reweighted line fitting of corridor boundaries and centerline construction 56

7.1 Physical mobile robot platform and front camera mounting setup . 68
7.2 ROS 2 software architecture node graph and topic communication flow 70
7.3 Execution time breakdown of the synchronous perception-control cycle on NVIDIA Jetson 76

8.1 Visual comparison of segmentation outputs on lettuce test images and on tomato and cucumber images 80
8.2 Comparative evaluation of geometric reference extraction across the four evaluated algorithms 81
8.3 Lateral tracking error and heading error profiles during inter-row autonomous navigation 86

Chapter 1

Introduction

▶ **DA COMPLETARE: Contesto di tesi e motivazione applicativa**

Inquadrare l'attività svolta nell'ambito della tesi di laurea magistrale:

- Collaborazione con il gruppo di ricerca di riferimento (Politecnico di Torino).
- Esigenze dell'agricoltura di precisione: diserbo meccanico selettivo, monitoraggio delle colture e riduzione dell'impatto ambientale senza dipendenza da segnali GNSS RTK sotto chioma o in serra.
- Obiettivo di realizzare una soluzione autonoma, modulare e fruibile su calcolatori embedded a basso costo (famiglia NVIDIA Jetson).

1.1 Context and motivation

Global population growth and rising food demand exert increasing pressure on agricultural productivity, while the sector faces persistent labor shortages and rising operating costs [1, 2]. Addressing these demands requires higher automation in field operations such as weeding, spraying, and crop monitoring. Row-based planting represents the dominant arrangement in modern farming, accounting for approximately 75% of cultivated land in structured agriculture [3]. Autonomous ground vehicles designed for these settings must navigate along inter-row corridors reliably without damaging crops or compacting productive soil beds.

Outdoor agricultural navigation historically relied on Global Navigation Satellite Systems (GNSS) augmented by Real-Time Kinematic (RTK) corrections. However, satellite positioning presents well-documented limitations in field environments: thick crop canopies cause multi-path reflections and severe signal attenuation, while greenhouse installations obstruct satellite reception entirely [1, 3]. Furthermore, RTK base station infrastructure adds cost. Alternative exteroceptive sensors such as 3D LiDAR provide direct spatial measurements, but their high unit cost hinders

economic scalability across commercial robotic fleets, and their point clouds lack the color and textural information required to distinguish crops from weeds [1, 3]. Monocular RGB cameras offer a cost-effective alternative that captures detailed chromatic and texture cues directly from the scene.

Early vision-based guidance pipelines extracted navigation lines through handcrafted image processing techniques. These approaches compute color vegetation indices to isolate plant matter from soil, apply intensity thresholding such as Otsu's method, and detect corridor edges using line-fitting algorithms or Hough transforms [1, 4]. Although computationally lightweight, these calibrated pipelines degrade when ambient field conditions shift. Variable sunlight angles, cast shadows from foliage, soil moisture gradients, specular reflections on wet leaves, and weed infestations frequently disrupt fixed thresholding rules and generate false boundary estimates.

Deep learning architectures, particularly convolutional neural networks for semantic segmentation, overcome these hand-tuning limitations. By learning hierarchical visual representations directly from image datasets, segmentation networks achieve higher tolerance to illumination shifts, ground irregularities, and weed clutter than handcrafted color models.

However, deep neural networks introduce a practical operational bottleneck: supervised training requires extensive pixel-level ground truth. Manually labeling agricultural scenes is labor-intensive and must cover multiple crop types, growth stages, and ambient conditions. In addition, running neural networks on compact onboard computers requires balancing segmentation accuracy against processing speed and power consumption, as steering commands must be updated fast enough to guide the vehicle in real time.

This thesis addresses these challenges through a modular pipeline that combines automatic dataset annotation with SAM 3, real-time onboard semantic segmentation, and deterministic geometric reference extraction for local robot guidance.

1.2 Objectives of the thesis

The main objectives of this work are:

- Evaluate whether segmentation networks trained on SAM 3 labels, used without manual correction, approach a network trained on manual masks, using manually annotated lettuce images as ground truth.

- Select a segmentation architecture balancing accuracy and inference speed for robotic deployment, through offline evaluation of FCN, DeepLabV3, and LR-ASPP on manually annotated lettuce images.

- Evaluate the effect of output label configurations (four-class, three-class, and binary path) on path segmentation accuracy.
- Implement an edge-based geometric extractor that converts segmentation masks into local image-space navigation references, and compare it with three baseline algorithms.
- Integrate the selected perception and geometric components on a robot platform for field validation.

1.3 Organization of the work

The thesis is organized as follows:

Chapter 2 introduces the agricultural navigation problem, the role of vision-based local guidance, and the segmentation-to-control pipeline structure.

Chapter 3 reviews vision-based methods for agricultural row navigation, covering classical and learning-based segmentation approaches, geometric extraction techniques, and experimental validation in related work.

Chapter 4 details the mathematical formulations and designs of the deep learning architectures evaluated in this work, focusing on FCN, DeepLabV3, MobileNetV3 with the LR-ASPP head, and SAM 3.

Chapter 5 describes dataset preparation with manual annotation and automatic SAM 3 labeling, empirical comparisons among the three architectures on lettuce, and evaluations of supervision strategies across label configurations.

Chapter 6 presents the geometric extractor that converts corridor masks into navigation references through horizontal-strip interval selection and independent edge fitting, together with three baseline algorithms used for comparison.

Chapter 7 describes the robot hardware, camera configuration, software integration using ROS, and the deployment of the trained segmentation model.

Chapter 8 reports offline and field validation results for segmentation accuracy, geometric reference extraction, and closed-loop navigation performance.

Chapter 9 summarizes the main findings, discusses limitations, and identifies directions for future work.

Chapter 2

Background and Problem Formulation

This chapter establishes the agricultural navigation context and formulates the vision-based local guidance problem addressed in this thesis.

2.1 Agricultural autonomous navigation

2.1.1 Navigation tasks and requirements

Agricultural robots perform tasks such as monitoring, spraying, harvesting, and weeding in structured crop environments [1]. Navigation between crop rows requires the robot to follow a path aligned with the row structure while avoiding plants and obstacles. The guidance reference must be computed from local sensing, as global GNSS positioning may be obstructed by crop canopies or greenhouse structures [1, 3].

Vision-based methods identify crop rows, inter-row paths, or crop boundaries from camera images and convert these observations into steering commands [1, 2]. The task is local: the robot maintains its position relative to the visible row structure, not a global map.

2.1.2 Environmental challenges

Field conditions vary with illumination, shadows, weather, growth stage, and weed presence [5, 2]. Plants may occlude parts of the path, and gaps or irregular spacing disrupt row structure. These changes affect both the appearance of image features and the reliability of geometric references extracted from them.

Segmentation models trained on one set of conditions must generalize to new lighting, crops, or row configurations. Geometric extractors must handle incomplete or ambiguous corridor observations without assuming perfect segmentation [6].

2.2 Vision-based local guidance

2.2.1 Sensing modalities

Camera-based perception provides appearance information for distinguishing plants, soil, and paths. Monocular RGB cameras are simpler and less expensive than LiDAR or stereo systems, but do not directly measure depth [1]. While RGB-D cameras provide both appearance and depth measurements, their performance depends on field conditions and hardware cost constraints.

This thesis uses monocular RGB sensing for local guidance. It does not reconstruct three-dimensional field structure or maintain a global map.

2.2.2 Image-space reference extraction

The guidance pipeline converts image observations into a reference suitable for control. Depending on the method, this may be a horizontal position, a fitted line, or a curve [7, 6, 4]. Image-space references describe the path geometry in pixel coordinates; they must be interpreted by the controller to generate velocity commands.

Straight-line models assume locally linear boundaries and produce simple references for proportional control [4]. They do not capture distant curves or complex row patterns, but are computationally efficient and suitable for short prediction horizons.

▶ DA COMPLETARE: Modello cinematico 4WS e formulazione analitica degli errori

Integrare la formalizzazione matematica del veicolo e del controllo:

- Modello cinematico per piattaforma 4WS a sterzatura simmetrica in controfase ($\delta_f = -\delta_r$) e relazione cinematica: $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$, $\dot{\theta} = \omega = \frac{2v\tan\delta}{L}$.
- Trasformazione rigida e proiettiva tra coordinate pixel (u, v) nel piano immagine e coordinate metriche nel frame robot $\{R\}$.
- Definizione rigorosa dell'offset laterale e_y e dell'heading error e_θ calcolati rispetto alla retta di mezzeria stimata.
- Formulazione della legge di guida proporzionale / Stanley per la determinazione della velocità angolare $\omega = K_p e_y + K_\theta e_\theta$.



Figure 2.1: Agricultural inter-row environment, vehicle frames, and geometric error definition

2.3 The segmentation-to-control pipeline

[SPAZIO FIGURA / SCHEMA]

Complete vision-based local guidance pipeline from camera stream to actuator commands

Schema a blocchi dell'architettura di navigazione: flusso continuo dall'acquisizione delle immagini monoculari, attraverso la rete di segmentazione semantica, l'estrazione geometrica dei bordi del corridoio, il calcolo degli errori di guida e la generazione dei comandi di velocità (v, ω).

Figure 2.2: Complete vision-based local guidance pipeline from camera stream to actuator commands

2.3.1 Perception stage

The perception stage assigns semantic labels to image pixels. For inter-row navigation, target classes typically comprise crop plants, the traversable path, covering material such as plastic mulch, and background [6]. The definition of output classes directly determines what the geometric stage can extract: a binary vegetation mask identifies crop rows, whereas a path mask isolates the drivable space between them [2, 3].

Segmentation models are trained on annotated field images. While transfer learning from pretrained backbones reduces training duration and improves feature generalization, the classification head must be tailored to the specific target crop classes and deployed at low latency on embedded hardware [3].

2.3.2 Geometric extraction stage

The geometric stage reduces the dense segmentation mask to a low-dimensional guidance reference for robot steering. Existing methods differ in their spatial search strategy and fitted geometric model: some aggregate mask pixels across image columns [4, 3], whereas others extract local intervals from horizontal strips [6] or search for line candidates within bounded angular regions [7].

The extraction algorithm must tolerate segmentation noise, missing plants, and cast shadows without producing erratic steering targets. Validation checks can reject estimates that fail geometric consistency thresholds, requiring the controller to handle transient frames where no valid reference is recovered [5].

2.3.3 Control integration

The controller converts the extracted geometric reference into forward velocity and steering commands. Proportional and look-ahead laws regulate steering using the lateral displacement and heading orientation errors relative to the image center [4].

In field operations, raw error signals extracted from semantic segmentation exhibit high-frequency jitter and occasional impulse disturbances caused by irregular foliage contours or transient detection dropouts. Directly feeding unfiltered errors into the steering loop induces actuator chattering and abrupt steering adjustments that compromise vehicle stability. Consequently, the control interface incorporates error conditioning structured across three complementary mechanisms:

1. **High-frequency smoothing:** a recursive low-pass filter attenuates persistent image-space tracking jitter without introducing excessive phase delay during row transitions;

2. **Impulse outlier rejection:** large step variations between consecutive frames are evaluated over temporal windows to isolate single-frame false detections while preserving genuine trajectory adjustments;

3. **Temporal hold and deadband:** a zero-order hold maintains command continuity across short-term perception dropouts, while a symmetric deadband around the corridor center prevents steering hunting.

The mathematical formulation and parameter tuning of this conditioning pipeline on the Demetra robotic platform are detailed in Section 7.2.1 of Chapter 7.

Closed-loop field evaluation measures the robot's ability to maintain row alignment under operational conditions. High segmentation accuracy and accurate geometric fitting are necessary prerequisites, but stable inter-row tracking ultimately depends on real-time execution and disturbance rejection over continuous runs [4, 2, 3].

Chapter 3

State of the Art

This chapter reviews vision-based methods for agricultural row navigation, focusing on the conversion of image observations into a local guidance reference. As surveyed by Bai et al. [1], vision sensors provide rich appearance and geometry cues for autonomous field guidance at low hardware cost. The review covers visual perception, geometric extraction, and the integration of these stages with robot control. Dataset preparation and the implementation used in this thesis are described in Chapters 5 and 6.

3.1 Classical image-processing methods

Classical pipelines combine image filtering, thresholding, and application-specific geometric rules. Mendez et al. convert vineyard images to grayscale, apply Gaussian filtering and Otsu thresholding, select the largest blob, and use column pixel counts to estimate the row center [4, Section 3.4]. Such methods can be effective in the conditions for which their parameters were selected, but changes in illumination, weeds, and crop appearance can require retuning [1].

3.2 Learning-based segmentation for agricultural navigation

3.2.1 Semantic segmentation approaches

Semantic segmentation assigns a class to each pixel, but the label definition determines what the geometric stage can recover. De Silva et al. train U-Net to predict narrow crop-row masks in sugar-beet images. Their labels represent row structure rather than individual plant pixels or traversable soil [2, Sections 3.3 and 4.1]. Navone et al. instead predict a binary vegetation mask from RGB images

using a MobileNetV3 backbone and an LR-ASPP head, then combine that output with depth for reference extraction [8, 3].

Like other path-oriented approaches in the literature, this work segments the traversable soil corridor between crop rows rather than the plants themselves. The segmentation model provides a candidate region of drivable ground, and the downstream geometric stage then extracts the lateral boundaries and computes the steering reference.

3.2.2 Training data and generalization

Pixel-level annotation is time-consuming, and agricultural datasets must represent changes in crop, growth stage, and lighting. De Silva et al. include 11 field variations in their dataset [2, Section 3]. Navone et al. initialize their segmentation model with ImageNet weights and train using synthetic AgriSeg images supplemented by 100 real crop images [3, Section 4.1, Table 1]. Pretraining and the choice of training images are separate design choices.

3.3 Semantic segmentation architectures

3.3.1 FCN, DeepLabV3, and mobile architectures

Fully Convolutional Networks (FCN) replace fully connected layers with convolutions to produce dense pixel-level predictions [9]. DeepLabV3 incorporates atrous convolutions and atrous spatial pyramid pooling to capture multi-scale context without decreasing feature resolution [10]. For embedded applications where compute resources are constrained, Howard et al. [11] introduced MobileNetV3 together with the Lite Reduced Atrous Spatial Pyramid Pooling (LR-ASPP) head, which reduces latency by using depthwise separable convolutions.

The choice of architecture affects both segmentation accuracy and inference speed. Deeper backbones or larger receptive fields can yield higher IoU values at the cost of additional compute. For robotic systems operating in closed loop, inference latency on edge hardware dictates the achievable control frequency.

3.3.2 Transfer learning and pretraining

Transfer learning initializes network weights from models pretrained on large visual corpora such as ImageNet. The pretrained backbone is a generic feature extractor, while the task-specific segmentation head is adapted to produce class scores matching the target dataset. In agricultural vision, pretraining reduces training time and mitigates overfitting when domain-specific annotated datasets are limited in size. For instance, Li et al. [6, Section 2.2.2] demonstrate that transferring

weights learned on corn fields to other crops (such as tomatoes, cucumbers, and wheat) preserves segmentation accuracy with small training sets. Similarly, Navone et al. [3, Section 4.1] initialize their segmentation backbone with ImageNet weights prior to fine-tuning on tree-row imagery.

The extent of pretraining differs across implementations. Some setups retain pretrained segmentation heads and replace only the final classification layer, whereas others initialize only the backbone and train the segmentation head from scratch.

3.4 Dataset preparation and model-assisted annotation

3.4.1 Manual annotation challenges

Manual pixel-level annotation requires annotators to label every pixel in training images. For agricultural datasets spanning multiple crops, growth stages, and lighting conditions, this process is time-consuming and expensive. Annotation consistency depends on annotator interpretation of ambiguous boundaries, particularly where plants overlap soil or covering material.

3.4.2 SAM and foundation models for segmentation

Foundation models for segmentation have established model-assisted annotation as an effective alternative to purely manual labeling. The Segment Anything project built its dataset with a model-in-the-loop data engine in three stages (assisted-manual, semi-automatic, and fully automatic), obtaining more than one billion masks on 11 million images [12]. While early foundation models required spatial prompts such as bounding boxes or foreground/background clicks for every target instance, Segment Anything Model 3 (SAM 3) introduces open-vocabulary concept segmentation driven directly by textual descriptions and visual exemplars [13].

In precision agriculture, model-assisted annotation with foundation models has been explored primarily for satellite-based field boundary delineation and canopy phenotyping. However, its systematic application to agricultural ground navigation remains largely unexplored. In this thesis, SAM 3 is employed offline as a zero-shot concept annotator, and its masks are used without manual correction. On lettuce, networks trained on SAM 3 labels with different class sets (path, plants, plastic mulch) are compared against manual ground truth; binary path labels generated for the complete lettuce, tomato, and cucumber collection train the network deployed on the robot. This offline annotation pipeline decouples the heavy computational burden of foundation models from the real-time inference requirements of the onboard robot controller.

3.5 Geometric extraction methods

The geometric stage reduces a mask to a quantity the controller can use. Depending on the representation and method, the output may be a horizontal image position, a straight line, a crop-edge estimate, or a curve. These outputs describe different tasks and coordinate systems.

3.5.1 Line fitting and crop-edge detection

Chen et al. estimate a path line for greenhouse cucumber rows from navigation points selected in plant–soil segmentation. Their prediction-point Hough transform uses a least-squares estimate to restrict the line search and is compared with traditional Hough and least-squares fitting [7, Sections 2–3].

Kneip et al. detect crop–ground transitions in scanlines of a stereo elevation map, then fit a linear crop-edge model using a Huber M-estimator and iteratively reweighted least squares. Their output is a crop boundary in ground-plane coordinates, not an image-space estimate of the inter-row path [5, Sections 6–7].

3.5.2 Column-wise histogram methods

Histogram methods aggregate mask pixels by image column. Mendez et al. estimate the row center from the five columns with the largest pixel counts in a thresholded background region [4, Section 3.4.3]. Navone et al. use a different input: a temporally combined vegetation mask refined with depth. SegMin smooths its column histogram and selects the minimum; SegMinD weights the vegetation mask by normalized inverse depth before calculating the histogram [3, Section 3]. In both cases the output is a horizontal position, not a full path line.

3.5.3 Horizontal strips and adaptive multi-ROI methods

Li et al. combine a Faster-U-net inter-row-path mask with an adaptive multi-ROI procedure. Horizontal regions supply ordered local points, which are used as control points for a multisegment cubic B-spline. Their experiments use 40-pixel search regions and discuss the trade-off between the number of points and fitting behavior [6, Section 2.2.3, Figure 9].

This is the closest geometric comparison for the present work because both pipelines select local evidence from horizontal image regions. The representation and fitted model differ: Li et al. use an inter-row-path mask and a spline, whereas this thesis estimates a local straight reference from the soil corridor between parallel crop rows.

3.5.4 Triangle Scan Method

De Silva et al. propose the Triangle Scan Method (TSM) for crop-row masks. It first scans an upper horizontal band to find an anchor column, using a preset anchor if the scan fails a threshold. Candidate lines from that anchor to positions along the base of a triangular region are then evaluated; the line with the greatest mask support defines the central crop row. Its two endpoints provide position and angular errors for visual servoing [2, Section 4.2, Algorithm 1].

TSM is not the same as extracting one point from each of several horizontal strips: the upper scan establishes an anchor for a bounded line search. It also operates on crop-row masks, so its output is not directly equivalent to a reference extracted from a soil mask.

3.5.5 Bilateral boundary tracking versus centroid reduction

In agricultural vision pipelines that convert binary segmentations into geometric references, existing methods fall into distinct categories, each characterized by specific operational trade-offs:

1. **Crop-canopy feature extraction with RANSAC:** A classical paradigm detects feature points along crop canopies or plant stalks, partitions them into left and right row clusters, and fits linear models using Random Sample Consensus (RANSAC) combined with ordinary least squares [1]. Recent monocular vision implementations extract crop row anchor points via DBSCAN clustering followed by RANSAC fitting to generate navigation lines, with reported angular and lateral errors minimized on the central crop row [14]. While functional in dense, uniform vegetation, this approach depends on continuous plant growth and degrades when seedlings are missing or canopy density is irregular. In addition, RANSAC is non-deterministic: random hypothesis generation introduces frame-to-frame angular jitter between consecutive video frames, requiring downstream temporal filtering to stabilize steering commands. The stochastic nature of RANSAC stems from its iterative random sampling strategy: each iteration selects a minimal subset of points (typically two for line fitting), computes a candidate model, and counts inliers within a residual threshold. To achieve 99% confidence of finding an all-inlier sample for a two-point linear model when outliers constitute 50% of the data, the algorithm requires 16 iterations, rising to 35 iterations when the outlier ratio reaches 65%; this sample count grows exponentially as outlier contamination increases [15]. On embedded platforms operating at 20 Hz with strict latency budgets, the computational overhead and output variance of RANSAC present practical limitations for closed-loop control.

2. **Global contour tracking with line fitting:** An alternative approach applies edge operators (such as Canny filtering) or boundary tracing algorithms (`findContours`) over the full segmented corridor mask, separating the resulting pixel cloud into lateral boundaries before fitting lines with RANSAC. On dense and noisy boundary masks, this introduces variable computational latency and retains the stochastic jitter inherent to random sampling, confirming that raw RANSAC output on monocular imagery lacks the determinism required for smooth steering commands.

3. **Horizontal strip decomposition with centroid reduction:** Methods operating on segmented drivable soil corridors (such as the multi-ROI scheme of Li et al. [6]) partition the image into horizontal bands and extract the bounding interval $[x_L, x_R]$ within each strip. However, these methods immediately collapse each interval into its midpoint:

$$x_C = \frac{x_L + x_R}{2} \tag{3.1}$$

   fitting curves (such as cubic B-splines or polynomials) directly to the sequence of centroids x_C. This premature reduction discards the distinction between the two physical edges. Under asymmetric disturbances—such as protruding leaves, localized weeds, or shadows extending along one side of the corridor—the midpoint shifts toward the intrusion, which introduces systematic bias in both lateral offset and heading angle.

4. **Cascaded deep learning for line extraction:** Recent studies investigate two-stage deep architectures, where an initial segmentation model is followed by a secondary network dedicated to semantic line detection. While capable of learning line features directly from data, running two cascaded neural networks on embedded platforms exceeds the memory and latency limits required for closed-loop robotic control (≥ 20 Hz).

5. **Single-edge elevation fitting:** M-estimators have also been applied to crop boundaries, such as Huber-loss Iteratively Reweighted Least Squares (IRLS) on stereo elevation scanlines [5]. However, these formulations track a single cutting boundary along dense harvesting fronts, rather than parallel crop boundaries defining an inter-row corridor from monocular RGB imagery.

To address the trade-off between computational determinism and disturbance decoupling without cascading neural networks, this thesis investigates a decoupled dual-edge linear formulation:

$$x_L(y) = m_L y + q_L, \qquad x_R(y) = m_R y + q_R \tag{3.2}$$

Both boundaries are estimated by Iteratively Reweighted Least Squares (IRLS) with the weight rule

$$w_i = \frac{1}{1 + \left(\frac{r_i}{\sigma}\right)^2} \tag{3.3}$$

applied for a fixed number of four iterations to at most one boundary sample per horizontal strip ($N = 24$ by default); Chapter 6 details how `numpy.polyfit` applies these weights. A perturbation on one boundary changes only the weights and fit of that boundary. The centerline reference $x_C(y) = (x_L(y) + x_R(y))/2$ is computed after both boundary fits.

3.5.6 Comparison of geometric representations

Table 3.1 summarizes the input representation and geometric output of the methods discussed above.

Table 3.1: Perceptual inputs and geometric references in selected agricultural navigation studies.

Study	Perceptual input	Geometric operation	Output
Chen et al. [7]	Plant–soil segmentation and navigation points	Restricted Hough search around a predicted point	Image-space path line
Kneip et al. [5]	Stereo elevation map	Scanline transitions and IRLS linear fit	Ground-plane crop edge
Mendez et al. [4]	Thresholded background region	Column counts and mean of the five largest	Horizontal row-center position
Navone et al. [3]	Vegetation mask with depth	Smoothed column histogram; inverse-depth weighting in SegMinD	Histogram-minimum position
Li et al. [6]	Predicted inter-row-path mask	Adaptive horizontal regions and cubic B-spline	Curved image-space path
De Silva et al. [2]	Predicted crop-row mask	Upper anchor and triangular line search	Central crop-row line
This thesis (Proposed DELE)	Predicted soil-corridor / mulch mask	Discrete horizontal strips, separate IRLS fits of the two boundaries	Straight image-space centerline $x_C(y)$ and (e_y, e_θ)

The methods are not directly ranked by this comparison: their inputs and outputs differ. Li et al. provide the closest structural precedent for strip-based point selection, while TSM illustrates a line search for a crop-row representation.

Neither method uses the same soil-corridor mask and straight-line model as the pipeline developed in this thesis.

3.6 System integration and experimental validation

3.6.1 Perception and control integration

Navigation systems use the extracted reference to generate motion commands. Navone et al. convert the displacement of their histogram reference into linear and angular velocity commands and smooth them with an Exponential Moving Average [3, Section 3.4]. Mendez et al. use proportional control based on row-center displacement and the distribution of background pixels [4, Section 3.5]. De Silva et al. combine the central row's angular and positional errors in a proportional visual-servoing controller for simulation [2, Section 4.3].

3.6.2 Scope of reported evaluations

The studies evaluate different stages of navigation. Kneip et al. assess crop-edge detection on recorded harvesting data [5, Section 8]; Chen et al. report angular errors and processing times for image-based path fitting [7, Sections 3.6, Tables 6–7]. De Silva et al. evaluate row detection on real images and visual servoing in simulation, with preliminary real-world row following also reported [2, Sections 5.2–5.3 and 6, Table 5]. Mendez et al. report closed-loop vineyard trials, measuring centering error from the distances to the two sides of the row [4, Sections 3.6 and 4]. Navone et al. test in simulation and crop fields, using LiDAR and attitude measurements to assess displacement and heading [3, Sections 4.3–4.4].

Segmentation IoU, image-space line error, field tracking error, and processing time measure different parts of the system. Results from these studies therefore provide context, not a common performance benchmark.

3.7 Positioning of this thesis

The system developed in this thesis addresses autonomous navigation in GNSS-denied crop rows using monocular RGB vision and embedded computation. Its technical positioning relative to the state of the art rests on three architectural decisions:

1. **Perceptual representation (Soil corridor versus canopy):** Rather than detecting crop stalks or foliage masks, the segmentation stage targets the

navigable soil corridor (and plastic mulch). This makes the navigation reference independent of crop growth stage, canopy height, and missing plants (fallanze), providing a continuous geometric guide directly at ground level.

2. **Geometric extraction (Decoupled deterministic boundaries versus centroid reduction):** Rather than collapsing strip intervals into midpoints prior to fitting (as in Li et al. [6]) or employing stochastic RANSAC over dense edge contours, the proposed pipeline decouples the left and right physical corridor boundaries. Each boundary is fitted independently by iteratively reweighted least squares with a fixed number of four iterations. A disturbance on one side (such as a leaf overhang or a shadow) enters only the fit of that boundary; the centerline, as the mean of the two edge lines, receives half of that boundary's deviation.

3. **Embedded deployment feasibility:** The architecture decouples offline automatic dataset generation from onboard execution. Large foundation models (SAM 3) are restricted to offline zero-shot concept annotation, while real-time inference on the NVIDIA Jetson Orin Nano is handled by a lightweight LR-ASPP network (using a MobileNetV3 backbone) paired with the deterministic IRLS geometric extractor. The entire pipeline executes in closed loop at 20 Hz without requiring secondary neural networks, stereo elevation mapping, or LiDAR sensors.

The chapters that follow detail and evaluate each stage of this pipeline: Chapter 4 formulates the deep learning architectures, Chapter 5 analyzes the annotation protocols and comparative model training, Chapter 6 formalizes the geometric extractors, Chapter 7 describes the Demetra robotic platform and onboard ROS integration, and Chapter 8 reports offline benchmarks and field tests.

Chapter 4

Deep Learning Architectures for Semantic Segmentation

In computer vision, semantic segmentation is the task of classifying every pixel in an image according to the category of the surface or object it belongs to [9], [16, Chapter 12]. Unlike image-level classification, which predicts a single global label for the entire scene, or object detection, which localizes entities with coarse bounding boxes, semantic segmentation generates a dense, pixel-level partition of the scene.

In this thesis, autonomous row navigation relies on a monocular vision pipeline in which semantic segmentation is the only perception stage. Rather than relying on satellite positioning (RTK-GPS) or active range sensors (LiDAR), the navigation system extracts the vehicle guidance reference directly from single RGB images by segmenting the navigable soil corridor from surrounding crop canopies, weed patches, and plastic mulch films under variable outdoor illumination.

To investigate the trade-off between segmentation accuracy, model complexity, and real-time execution on the mobile robot, this chapter analyzes three distinct deep learning architectures:

- **Fully Convolutional Networks (FCN):** pairing a dilated ResNet-50 backbone with a convolutional segmentation head (`FCNHead`);
- **DeepLabV3:** pairing a dilated ResNet-50 backbone with an Atrous Spatial Pyramid Pooling (ASPP) head;
- **MobileNetV3 with LR-ASPP:** pairing a MobileNetV3-Large backbone with a Lite Reduced ASPP head.

In addition, the Segment Anything Model 3 (SAM 3) is introduced as an offline foundation model for semi-automated training dataset annotation.

The remainder of this chapter is structured as follows. Section 4.1 establishes the mathematical formulation and encoder-decoder principles. Sections 4.2, 4.3, and 4.4 detail the three segmentation networks, while Section 4.5 describes the SAM 3 annotation pipeline.

4.1 Problem formulation and encoder-decoder principles

Let $I \in \mathbb{R}^{H\times W\times 3}$ denote an input color image with height H and width W. Semantic segmentation aims to estimate a dense label map $Y \in \{1,\dots,C\}^{H\times W}$, where each pixel (i,j) is assigned to one of C mutually exclusive semantic classes [9], [16, Chapter 12]. Deep segmentation networks approximate this mapping through a parameterized function $f_\theta(I)$ that outputs an unnormalized score tensor $Z \in \mathbb{R}^{H\times W\times C}$ containing the class logits for every pixel location. The predicted probability of class c at pixel (i,j) is obtained by applying a pixel-wise softmax over the class dimension:

$$p_{i,j,c} = \frac{\exp(Z_{i,j,c})}{\sum_{k=1}^{C}\exp(Z_{i,j,k})}. \tag{4.1}$$

During supervised training, network parameters θ are optimized by minimizing a loss function $\mathcal{L}(Y^*,p)$ that quantifies the discrepancy between the predicted probability map p and the ground-truth annotations $Y^* \in \{0,1\}^{H\times W\times C}$, represented in one-hot form [9]. Through backpropagation, the gradients of this loss with respect to θ guide stochastic optimization algorithms (such as SGD or AdamW) to iteratively update the network weights. Because semantic segmentation operates densely across the spatial grid, the overall loss is evaluated by aggregating individual prediction errors over all pixel locations (i,j) [9], [16, Chapter 12].

A widely used training objective for semantic segmentation is the pixel-wise categorical cross-entropy loss, averaged across all pixels:

$$\mathcal{L}_{\mathrm{CE}}(Y^*,p) = -\frac{1}{HW}\sum_{i=1}^{H}\sum_{j=1}^{W}\sum_{c=1}^{C} Y^*_{i,j,c}\log(p_{i,j,c}). \tag{4.2}$$

For each pixel, this formulation penalizes the negative logarithm of the probability assigned to the ground-truth category: the penalty approaches zero when the network predicts the correct class with high confidence ($p_{i,j,c} \approx 1$), and grows large when the assigned probability is low ($p_{i,j,c} \to 0$).

In practical segmentation tasks, loss functions can be adapted to handle specific domain challenges. When target classes occupy highly disproportionate image areas, cross-entropy can be extended with class-specific penalty weights w_c to mitigate the dominance of majority classes during optimization [9], as formulated for our

agricultural dataset in Chapter 5. Other common alternatives in the literature include region-based losses (such as the Dice loss) that optimize spatial segment overlap, and boundary-aware losses designed to refine object contours.

The output stride S denotes the ratio of the input image spatial resolution to the spatial dimensions of the extracted feature map [10]. Many classification backbones progressively reduce spatial resolution through consecutive strided convolutions and pooling operations, reaching an output stride of up to $S = 2^5 = 32$ [16, Chapter 12]. While spatial downsampling expands the effective receptive field and enables deeper layers to extract abstract semantic features (determining *what* is depicted in the scene), it discards fine-grained spatial coordinates (obscuring *where* object boundaries lie at the pixel level). Segmentation backbones may instead employ smaller output strides, such as $S = 8$ or $S = 16$, using atrous convolutions to preserve denser feature representations [10].

To resolve this trade-off between high-level semantic abstraction and pixel-level spatial precision, modern semantic segmentation models commonly adopt an encoder-decoder architecture:

- **Encoder (Backbone):** A sequence of feature-extraction stages that produces high-level semantic representations, yielding a feature tensor $F_{\text{enc}} \in \mathbb{R}^{\frac{H}{S} \times \frac{W}{S} \times C_{\text{enc}}}$, where S denotes the encoder output stride. Depending on the architecture, intermediate feature maps at different resolutions may also be supplied to the decoder.
- **Decoder and segmentation head:** The decoder processes the encoded features and progressively recovers spatial resolution, optionally combining them with higher-resolution encoder features through skip connections [9]. A final segmentation head maps the resulting features to class logits, which are upsampled to the input image resolution when necessary.

4.2 Fully Convolutional Networks (FCN)

Traditional Convolutional Neural Networks are designed for image-level classification, producing a single label for the entire input. Fully Convolutional Networks (FCN), introduced by Long et al. [9], adapt these architectures for semantic segmentation by predicting a class label for every individual pixel in an end-to-end manner.

This is achieved through convolutionalization: replacing the final dense (fully connected) layers with convolutional filters allows the network to process images of arbitrary size and output dense spatial score maps. In this study, the FCN architecture is instantiated with a ResNet-50 [17] residual backbone and an `FCNHead` segmentation head, matching the standard TorchVision implementation (`fcn_resnet50`).

4.2.1 Convolutionalization of classification networks

In standard classification networks, the feature map produced by the final convolutional stage is flattened into a one-dimensional vector and fed into dense (fully connected) layers. Because a dense layer contains a fixed number of weights dedicated to each flattened input position, the network is restricted to a rigid, predetermined input image resolution.

As detailed by Géron [16, Chapter 12], a dense layer with C units connected to an input feature map of spatial dimension $h \times w$ with C_{in} channels can be replaced by a convolutional layer with C filters, a filter size equal to $h \times w$, and *valid* padding. The convolutional layer then outputs a $1 \times 1 \times C$ tensor containing the same values produced by the dense layer.

Because a convolution is applied at every spatial position, the convolutionalized network no longer requires a fixed input size. When the input feature map is larger than $h \times w$, the same filters slide over it and output a spatial grid of class scores in a single forward pass. This result is equivalent to sliding the original classification network across the image, but each intermediate activation is computed only once instead of once per overlapping window [16, Chapter 12].

4.2.2 Skip architecture and upsampling

The classification networks adapted by Long et al. (AlexNet, VGG-16, and GoogLeNet [9]) apply an overall stride of $S = 32$, so the final score map is 32 times smaller than the input image [16, Chapter 12]. Bilinear interpolation works reasonably well only for upsampling factors up to $4\times$ or $8\times$ [16, Chapter 12]; a single $32\times$ interpolation step therefore cannot restore boundary detail.

For in-network upsampling, Long et al. [9] adopted transposed convolutions (also referred to as fractionally strided convolutions or deconvolutions, available in PyTorch as `nn.ConvTranspose2d`). A transposed convolution with stride s can be interpreted as expanding the input feature map by inserting $s - 1$ rows and columns of zeros between adjacent elements, followed by a standard convolution with filter weights $w \in \mathbb{R}^{K \times K}$ [16, Chapter 12]. Algebraically, the operation reverses the forward and backward passes of a convolution [9]. In the original FCN, the intermediate $2\times$ upsampling layers are initialized to bilinear interpolation and then learned, whereas the final upsampling layer is kept fixed to bilinear interpolation [9].

Upsampling directly from stride 32 in a single step (FCN-32s) yields coarse segmentation boundaries. To refine them, Long et al. introduced skip connections that fuse high-level semantic features from deep layers with fine-grained appearance information from shallower layers [9]:

- **FCN-32s:** Upsamples the final stride-32 score map directly to image resolution in a single $32\times$ step.

- **FCN-16s:** Upsamples the stride-32 score map by a factor of 2×, adds it element-wise to predictions computed from the stride-16 layer (`pool4` in VGG-16), and upsamples the combined map by 16×.
- **FCN-8s:** Upsamples the fused FCN-16s score map by 2×, sums it with predictions from the stride-8 layer (`pool3`), and upsamples the result by 8× back to $H \times W$.

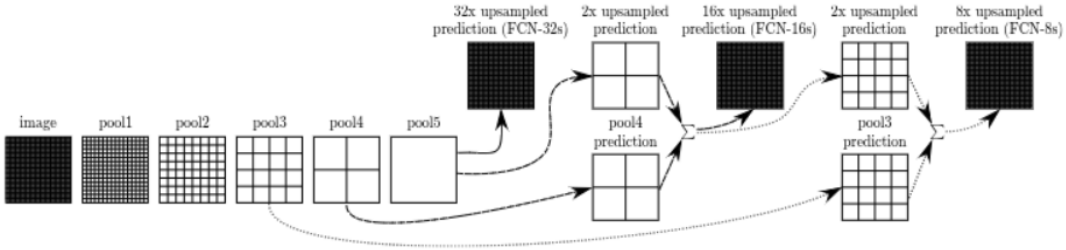


Figure 4.1: Fully Convolutional Network (FCN) architecture with progressive skip connections combining deep semantic features with shallow appearance representations (adapted from Long et al. [9]).

Implementation note: The experimental evaluation in this thesis employs the TorchVision `fcn_resnet50` implementation [18]. Unlike the original FCN-8s architecture, this model does not fuse predictions from multiple backbone stages. Instead, its ResNet-50 backbone replaces the spatial downsampling in `layer3` and `layer4` with dilated convolutions, yielding an output stride of 8. The main segmentation head (`FCNHead`) processes the 2048-channel features from `layer4` using a 3×3 convolution that reduces the channel dimension to 512, followed by batch normalization, ReLU activation, dropout with probability 0.1, and a 1×1 convolution that produces C class logits. The logits are then resized directly to the input image dimensions $H \times W$ using bilinear interpolation with `align_corners=False`. When enabled, an auxiliary classifier applies the same head structure to the 1024-channel features from `layer3`, also at output stride 8, using 256 intermediate channels. Its output can provide an additional supervision signal during training. The primary segmentation prediction is obtained from the main output (`out`), without fusion with the auxiliary output.

4.3 DeepLabV3 and atrous convolutions

DeepLabV3, developed by Chen et al. [10], addresses the resolution loss caused by consecutive pooling operations by replacing standard downsampling with atrous (dilated) convolutions and multi-scale pooling modules.

4.3.1 Atrous convolution formulation

Chen et al. [10] define atrous convolution in one dimension as $y[i] = \sum_k x[i+r\cdot k]\, w[k]$ (Eq. 1 in [10]), where the atrous rate r is the stride with which the input signal is sampled. Equivalently, the input is convolved with an upsampled filter obtained by inserting $r-1$ zeros between two consecutive filter values along each spatial dimension [10]. Applied along both spatial dimensions to a 2D feature map $x[i,j]$ with a $K \times K$ filter $w[p,q]$, where $p, q \in \{-M, \ldots, M\}$ and $M = \frac{K-1}{2}$ for odd K, the operation becomes:

$$y[i,j] = \sum_{p=-M}^{M} \sum_{q=-M}^{M} x[i + r \cdot p,\ j + r \cdot q] \cdot w[p,q]. \tag{4.3}$$

Standard convolution is the special case $r = 1$ [10]. Because $r-1$ zeros are inserted in each of the $K-1$ gaps between filter values, the effective filter size K' is:

$$K' = K + (K-1)(r-1). \tag{4.4}$$

Géron gives the one-dimensional case: the filter $[1, 2, 3]$ dilated with rate 4 becomes $[1, 0, 0, 0, 2, 0, 0, 0, 3]$, of length $3 + 2 \times 3 = 9$ [16, Chapter 12]. Likewise, a 3×3 filter with rate $r = 6$ covers a 13×13 region ($3 + 2 \times 5 = 13$) with 9 learnable parameters. Atrous convolution therefore enlarges the receptive field without extra parameters [16, Chapter 12] and without reducing the spatial resolution of the feature maps [10].

4.3.2 Atrous Spatial Pyramid Pooling (ASPP)

Objects and agricultural corridors appear at varying visual scales depending on camera mounting angle and distance from the vehicle. DeepLabV3 captures contextual information across multiple scales using the Atrous Spatial Pyramid Pooling (ASPP) module.

The ASPP module operates in parallel on the high-level feature map extracted by the backbone and consists of five branches:

1. One 1×1 standard convolution with 256 filters and batch normalization.
2. Three 3×3 atrous convolutions with 256 filters each, using dilation rates $r \in \{6, 12, 18\}$ for an output stride of 16 (or $r \in \{12, 24, 36\}$ for stride 8).
3. One Image-Level Pooling branch: to incorporate global image context, this branch computes global average pooling across the spatial dimensions of the input tensor, processes the resulting $1 \times 1 \times C_{\text{in}}$ vector with a 1×1 convolution (256 filters) and batch normalization, and upsamples the output bilinearly to the original ASPP spatial resolution.

Implementation note: The TorchVision `deeplabv3_resnet50` implementation [19] employs a dilated ResNet-50 backbone with output stride 8, achieved by removing downsampling in `layer3` and `layer4` and introducing atrous convolutions. The ASPP module correspondingly uses dilation rates $r \in \{12, 24, 36\}$ for the three parallel atrous branches.

The outputs of all five branches are concatenated along the channel dimension, resulting in a tensor with $5 \times 256 = 1280$ channels:

$$F_{\text{concat}} = [F_{1\times1},\ F_{\text{atrous},r_1},\ F_{\text{atrous},r_2},\ F_{\text{atrous},r_3},\ F_{\text{image_pool}}]\,. \tag{4.5}$$

This concatenated representation is projected back to 256 channels via a 1×1 convolution with batch normalization [10]; TorchVision follows it with ReLU activation and dropout ($p = 0.5$) [19]. The TorchVision implementation then applies a 3×3 convolution ($256 \to 256$ channels) with batch normalization and ReLU before the final 1×1 classification layer maps to C class logits. The output is upsampled via bilinear interpolation to image resolution $H \times W$. The ASPP parallel branch layout is depicted in Figure 4.2.

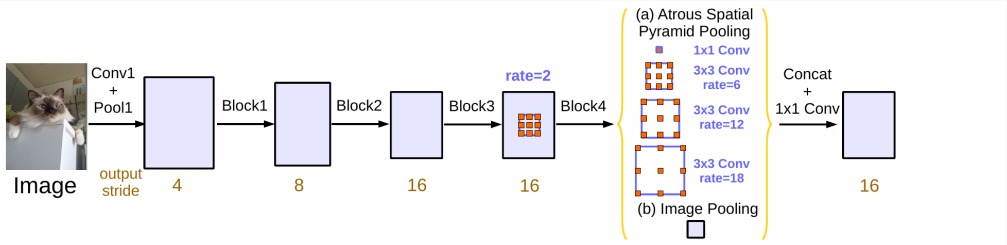


Figure 4.2: DeepLabV3 architecture featuring the Atrous Spatial Pyramid Pooling (ASPP) module with parallel atrous convolutions and image-level pooling (adapted from Chen et al. [10]).

4.4 MobileNetV3 and the Lite Reduced ASPP (LR-ASPP) head

DeepLabV3 with a ResNet-50 backbone provides high segmentation accuracy, but it is the slowest of the three networks in the offline comparison of Chapter 5 (86.87 ms mean recorded inference time). For mobile devices, Howard et al. [11] developed MobileNetV3, tuned to mobile phone CPUs, together with the Lite Reduced ASPP (LR-ASPP) segmentation head.

4.4.1 Depthwise separable convolutions

The building block that MobileNet architectures inherit from MobileNetV1 is the depthwise separable convolution [11], which factors a standard convolution into two separate steps: a spatial depthwise convolution and a channelwise pointwise convolution.

Consider an input feature map $X \in \mathbb{R}^{H \times W \times C_{\text{in}}}$ and an output feature map $Y \in \mathbb{R}^{H \times W \times C_{\text{out}}}$, processed with spatial filter size $D_K \times D_K$.

- **Standard convolution:** Each filter channel interacts with every input channel simultaneously. The computational cost, measured in Multiply-Accumulate (MAC) operations, is:

$$\text{Cost}_{\text{standard}} = H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot D_K^2. \tag{4.6}$$

- **Depthwise convolution:** Applies a single $D_K \times D_K$ spatial filter per input channel independently, without channel cross-talk:

$$\text{Cost}_{\text{depthwise}} = H \cdot W \cdot C_{\text{in}} \cdot D_K^2. \tag{4.7}$$

- **Pointwise convolution:** Applies C_{out} filters of size 1×1 across all channels to compute linear combinations of the depthwise outputs:

$$\text{Cost}_{\text{pointwise}} = H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}}. \tag{4.8}$$

The total cost of a depthwise separable convolution is the sum of Eqs. (4.7) and (4.8):

$$\text{Cost}_{\text{sep}} = H \cdot W \cdot C_{\text{in}} \cdot D_K^2 + H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}}. \tag{4.9}$$

Comparing Eq. (4.9) to Eq. (4.6) yields the computational reduction ratio:

$$\frac{\text{Cost}_{\text{sep}}}{\text{Cost}_{\text{standard}}} = \frac{H \cdot W \cdot C_{\text{in}} \cdot D_K^2 + H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}}}{H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot D_K^2} = \frac{1}{C_{\text{out}}} + \frac{1}{D_K^2}. \tag{4.10}$$

For 3×3 filters ($D_K = 3$), the ratio is $\frac{1}{64} + \frac{1}{9} \approx 0.13$ for $C_{\text{out}} = 64$ and approaches $\frac{1}{9} \approx 0.11$ as C_{out} grows. The parameter count follows the same ratio, since it equals the cost without the factor $H \cdot W$. Depthwise separable convolutions thus reduce arithmetic operations and parameters by about 87–89% relative to standard convolutions.

4.4.2 Inverted residual blocks with linear bottlenecks

In classical ResNet-50 bottleneck blocks, the input channels are first compressed via a 1×1 convolution, processed with a 3×3 convolution at reduced dimensionality, and

expanded back to the original high channel count (wide → narrow → wide). This structure reduces computational cost while maintaining representational capacity.

MobileNetV3 adopts the inverted residual block with linear bottleneck introduced by MobileNetV2, which reverses this design (narrow → wide → narrow) [11]:

1. **Expansion layer:** A 1×1 pointwise convolution expands the low-dimensional input C_{in} to a higher-dimensional space $e \cdot C_{\text{in}}$, where the expansion factor e varies between blocks (from 1 to 6 in MobileNetV3-Large [11, Table 1]).

2. **Depthwise layer:** A 3×3 or 5×5 depthwise convolution performs spatial filtering in the expanded channel space.

3. **Linear bottleneck:** A 1×1 projection layer compresses the channels back to C_{out}. No non-linear activation follows this projection: the narrow input and output of the block remain linear, and the non-linearities act only in the expanded space [11].

When the input and output have the same number of channels and the stride is 1, a residual shortcut connection adds the input directly to the block output, as illustrated in Figure 4.3a.

4.4.3 Squeeze-and-Excitation (SE) channel attention

To recalibrate channel responses, MobileNetV3 inserts Squeeze-and-Excitation (SE) modules in selected inverted residual blocks (8 of the 15 blocks of MobileNetV3-Large [11, Table 1]). The module is placed after the depthwise convolution, so that it acts on the expanded representation:

1. **Squeeze:** Global spatial average pooling aggregates the spatial dimensions of feature map $U \in \mathbb{R}^{H' \times W' \times C'}$ into a channel descriptor vector $z \in \mathbb{R}^{C'}$:

$$z_c = \frac{1}{H'W'} \sum_{i=1}^{H'} \sum_{j=1}^{W'} U_{i,j,c}. \tag{4.11}$$

2. **Excitation:** Two fully connected layers parameterized by weights $W_1 \in \mathbb{R}^{\frac{C'}{4} \times C'}$ and $W_2 \in \mathbb{R}^{C' \times \frac{C'}{4}}$ model non-linear channel relationships; the SE bottleneck is fixed to $1/4$ of the channels of the expansion layer [11]:

$$s = \sigma\left(W_2 \cdot \text{ReLU}(W_1 z)\right), \tag{4.12}$$

   where $\sigma(\cdot)$ denotes the hard-sigmoid activation function.

3. **Scale:** The input tensor U is rescaled channel-wise by the activation vector s:

$$\tilde{U}_{i,j,c} = s_c \cdot U_{i,j,c}, \qquad \forall c \in \{1, \ldots, C'\}. \tag{4.13}$$

4.4.4 Efficient non-linearities: hard-swish and hard-sigmoid

The swish non-linearity, defined as $\text{swish}(x) = x \cdot \sigma(x)$, improves the accuracy of neural networks when used as a drop-in replacement for ReLU. However, the sigmoid $\sigma(x) = (1 + e^{-x})^{-1}$ is much more expensive to compute on mobile devices and makes accuracy difficult to maintain in fixed-point arithmetic [11].

MobileNetV3 replaces standard sigmoid and swish with piece-wise linear hard approximations based on $\text{ReLU6}(x) = \min(\max(x, 0), 6)$:

$$\text{h-sigmoid}(x) = \frac{\text{ReLU6}(x+3)}{6}, \tag{4.14}$$

$$\text{h-swish}(x) = x \cdot \text{h-sigmoid}(x) = x \cdot \frac{\text{ReLU6}(x+3)}{6}. \tag{4.15}$$

Eqs. (4.14) and (4.15) contain no exponential. Howard et al. found no discernible difference in accuracy between the hard and smooth versions, while the piece-wise form reduces latency on mobile CPUs [11]. Because the cost of a non-linearity decreases as activation maps shrink, MobileNetV3 uses h-swish only in the second half of the network and ReLU in the earlier layers [11].

4.4.5 The Lite Reduced ASPP (LR-ASPP) architecture

The DeepLabV3 ASPP head applies four parallel convolutional branches with 256 channels each plus an image-pooling branch. Howard et al. proposed the Lite Reduced ASPP (LR-ASPP) head as a lighter alternative: it deploys global average pooling in a fashion similar to the Squeeze-and-Excitation module and uses only one 1×1 convolution in the pooling branch [11].

The LR-ASPP architecture uses features from two distinct stages of the MobileNetV3-Large backbone:

- A low-resolution, high-semantic feature map $F_{16} \in \mathbb{R}^{\frac{H}{16} \times \frac{W}{16} \times 960}$ extracted at output stride 16.
- A high-resolution, low-semantic feature map $F_8 \in \mathbb{R}^{\frac{H}{8} \times \frac{W}{8} \times 40}$ extracted at output stride 8.

The TorchVision LR-ASPP implementation (`lraspp_mobilenet_v3_large`) [20] processes the two feature streams as follows:

1. The stride-16 feature map F_{16} passes through two parallel branches:
   - **CBR path:** A 1×1 convolution followed by batch normalization and ReLU activation projects F_{16} to an intermediate representation $X \in \mathbb{R}^{\frac{H}{16} \times \frac{W}{16} \times C_{\text{inter}}}$, with $C_{\text{inter}} = 128$:

$$X = \text{ReLU}\left(\text{BN}\left(\text{Conv}_{1\times1}(F_{16})\right)\right). \tag{4.16}$$

- **Global context branch:** Applies global average pooling followed by a 1×1 convolution and a standard sigmoid activation to produce a channel-wise gating vector $G \in \mathbb{R}^{1\times1\times C_{\text{inter}}}$:

$$G = \sigma\left(\text{Conv}_{1\times1}\left(\text{GAP}(F_{16})\right)\right). \tag{4.17}$$

2. The CBR output X is modulated element-wise by the gating vector G and then upsampled by a factor of 2 via bilinear interpolation:

$$M_{16} = \text{BilinearUpsample}_{2\times}(X \odot G). \tag{4.18}$$

3. The upsampled context features M_{16} and the stride-8 features F_8 pass through **separate** 1×1 classification layers to produce class logits:

$$Z_{16} = \text{Conv}^{(16)}_{1\times1}(M_{16}), \qquad Z_8 = \text{Conv}^{(8)}_{1\times1}(F_8). \tag{4.19}$$

4. The two logit maps are **summed in logit space** to produce the final stride-8 prediction:

$$Z_{\text{fused}} = Z_{16} + Z_8. \tag{4.20}$$

5. A final $8\times$ bilinear upsampling step restores the prediction to the original image dimensions $H \times W$.

Implementation note: Unlike architectures that fuse feature representations before classification, LR-ASPP fuses predictions in logit space. This design allows each branch to learn independent classification boundaries while combining their complementary spatial resolutions. The global context gating uses a standard sigmoid activation (Eq. (4.17)), not the hard-sigmoid approximation employed in the MobileNetV3 inverted residual blocks.

By replacing the multi-branch parallel ASPP structure with a single global pooling gate and logit-space fusion, LR-ASPP reduces computational operations while retaining global scene context and boundary localization. The inverted residual backbone block and the complete LR-ASPP segmentation topology are illustrated in Figure 4.3.

4.5 Segment Anything Model 3 (SAM 3) for dataset annotation

Manually annotating agricultural images at pixel precision requires drawing polygons around irregular plant boundaries, inter-row soil, and covering material. The time required for this labeling motivated the use of SAM 3 to generate training

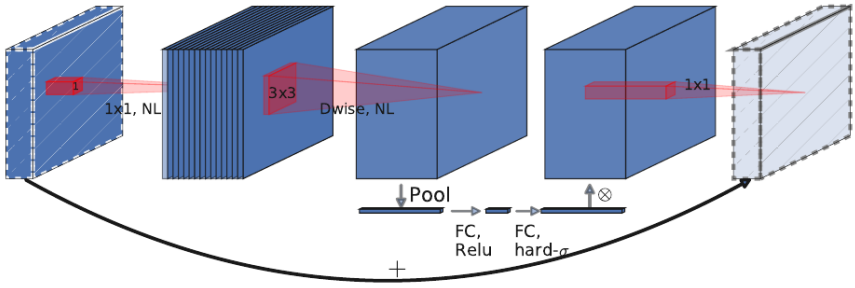


Figure 4. MobileNetV2 + Squeeze-and-Excite [20]. In contrast with [20] we apply the squeeze and excite in the residual layer. We use different nonlinearity depending on the layer, see section 5.2 for details.

(a) MobileNetV3 Inverted Residual block with Squeeze-and-Excitation channel attention.

V3-Small†	16.1	43	1.77	0.16

Table 6. Object detection results of SSDLite with different backbones on COCO test set. †: Channels in the blocks between $C4$ and $C5$ are reduced by a factor of 2.

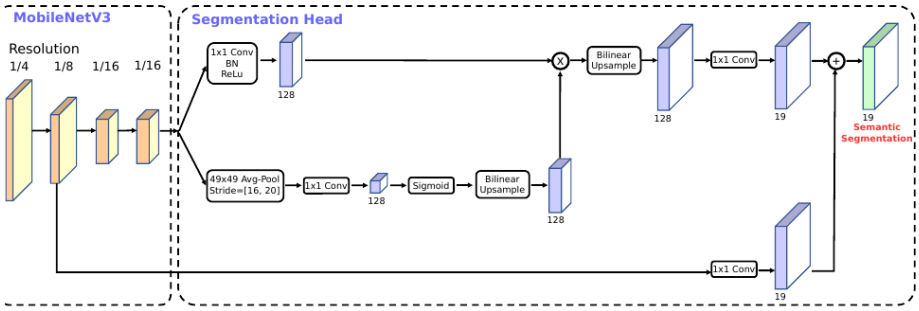


Figure 10. Building on MobileNetV3, the proposed segmentation head, Lite R-ASPP, delivers fast semantic segmentation results while mixing features from multiple resolutions.

(b) Lite Reduced ASPP (LR-ASPP) segmentation head mixing low- and high-resolution features.

Figure 4.3: MobileNetV3 and Lite Reduced ASPP (LR-ASPP) architecture: (a) inverted residual block combining depthwise separable convolution, linear bottleneck, and Squeeze-and-Excitation channel attention; (b) segmentation head combining stride-16 features modulated by global context gating with stride-8 fine features (adapted from Howard et al. [11]).

labels automatically (Chapter 5). Foundation models make it possible to generate candidate masks from prompts without training on the target crops.

Kirillov et al. [12] established the promptable segmentation paradigm with SAM, using Vision Transformers to generate binary segmentation masks from spatial prompts such as foreground/background points, bounding boxes, or coarse masks. Carion et al. [13] extended this paradigm in SAM 3 by introducing open-vocabulary concept segmentation driven by textual descriptions and visual exemplars.

4.5.1 Architecture overview

Unlike the original SAM architecture, which processes images through a single ViT encoder followed by a prompt-conditioned mask decoder, SAM 3 adopts a dual-architecture design comprising a **detector** for image-level segmentation and a **tracker** for video sequences. Both components share a unified **Perception Encoder (PE)** backbone that processes visual and linguistic inputs.

The SAM 3 detector follows a DETR-based paradigm with the following key components:

1. **Perception Encoder:** A shared vision-language backbone that encodes images into spatial feature representations. Unlike SAM's standalone ViT encoder, the PE is designed to handle both visual data and textual concept embeddings.

2. **Fusion Encoder:** Accepts unconditioned image embeddings from the PE and conditions them by cross-attending to prompt tokens derived from text descriptions (e.g., the prompts `"plants"`, `"central farm path"`, and `"plastic mulch film"` used in Chapter 5) or visual exemplars (bounding boxes around reference instances).

3. **Presence Token:** A learned global token, processed in the decoder together with the object queries, that decouples recognition from localization. It predicts only whether the prompted concept is present in the image, while each object query solves the localization problem; the final score of each query is the product of its own score and the presence score. Carion et al. report that this decomposition reduces the detector's sensitivity to false positives [13].

4. **Decoder:** Learned object queries cross-attend to the conditioned image embeddings from the fusion encoder, producing a set of binary segmentation masks and associated confidence scores for all instances of the target concept.

For video input, SAM 3 extends this architecture with a memory-augmented tracker that shares the frozen PE backbone with the detector. The tracker incorporates a memory encoder and memory bank that store object appearance across frames, enabling consistent temporal segmentation.

Figure 4.4 illustrates the original SAM architecture, which established the foundation for prompt-based segmentation; SAM 3 extends this paradigm with text-driven concept detection and video tracking capabilities.

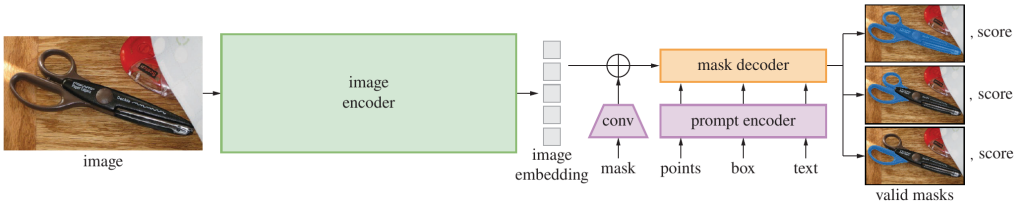


Figure 4.4: Original Segment Anything Model (SAM) architecture with ViT image encoder, prompt encoder, and mask decoder (adapted from Kirillov et al. [12]). SAM 3 extends this foundation with a dual detector-tracker architecture and open-vocabulary concept segmentation.

4.5.2 Operational distinction from onboard models

A key architectural distinction must be emphasized between the foundation model and the robotic segmentation network:

- SAM 3 has about 850 million parameters, of which about 450 million belong to the vision encoder and 300 million to the text encoder [13]. Its reported speed of 30 ms per image refers to an NVIDIA H200 data-center GPU [13], a hardware class not available on the robot, and the model was not evaluated on the onboard computer in this work.
- In this thesis, SAM 3 functions strictly as an **offline supervision generator**. Given text prompts representing crop and corridor concepts, SAM 3 generates dense candidate masks across lettuce, tomato, and cucumber image collections.
- The resulting masks are used without manual correction as training targets for LR-ASPP, the network selected for onboard perception after the comparison on the manually annotated lettuce dataset (Chapter 5). LR-ASPP recorded a mean offline inference time of 10.41 ms in that comparison.

4.6 Summary of architectural characteristics

Table 4.1 summarizes the characteristics of the three segmentation networks and of SAM 3 analyzed in this chapter, establishing the foundation for the dataset experiments and comparative evaluations presented in Chapter 5.

Table 4.1: Comparison of deep learning architectures used in this thesis. Recorded inference times are reported in Chapter 5.

Model	Backbone	Context mechanism	Target role	Primary characteristics
FCN	Dilated ResNet-50	Single-scale `FCNHead` on stride-8 features, auxiliary classifier during training	Benchmark segmentation	High parameter count, no multi-scale context module
DeepLabV3	Dilated ResNet-50	Multi-scale ASPP on stride-8 features	High-accuracy benchmark	Longest recorded inference time, multi-scale context
LR-ASPP	MobileNetV3-Large	Gated global pooling with logit-space fusion	Onboard segmentation for navigation	Compact architecture designed for mobile devices
SAM 3	Perception Encoder (vision-language)	DETR-based detector with presence token and text/visual prompts	Offline automatic annotation	Foundation model for open-vocabulary concept segmentation; not deployed onboard

Chapter 5

Dataset Preparation and Comparative Model Training

Following the theoretical analysis of deep learning architectures in Chapter 4, this chapter describes the datasets, annotation pipelines, and experimental model evaluations that guide the configuration for robotic integration.

The experimental study began by fine-tuning FCN, DeepLabV3, and LR-ASPP from pretrained weights on a manually annotated lettuce dataset for an empirical comparison. Because manual segmentation was time-consuming, this initial training was limited to lettuce rather than repeated across every target crop. Its outcome led to the adoption of LR-ASPP for embedded deployment.

The second experiment, carried out after the selection of LR-ASPP, tests whether networks trained on masks generated automatically by SAM 3, without any manual correction, can replace manual annotation. Manual masks were available for lettuce and were more accurate, but the aim was to evaluate labels that require no human work.

The experiment uses only lettuce images, because lettuce is the only crop with manual ground truth. SAM 3 generated three label sets for these images, with four, three, and two classes (Section 5.1.2), and a separate LR-ASPP network was trained on each. The predictions of each network are compared with the manual lettuce masks on all its classes. The path class matters most, because it is the only one used for row following. The SAM 3 masks themselves are not compared with the manual masks.

The binary labeling was then extended to the complete image collection, labeled entirely by SAM 3 (`Data_allPlantsSAM2`), and a network trained on this collection is the one deployed on the robot. The results of this configuration are reported in Table 5.7.

This chapter details image acquisition, dataset partitioning, training parameters,

and offline benchmark evaluations. Subsequent chapters detail geometric navigation extraction and closed-loop robot validation.

5.1 Image acquisition and class definition

5.1.1 Image sources and acquisition conditions

[SPAZIO FIGURA / SCHEMA]

Sample field images across crops and corresponding training masks

Esempi visivi delle acquisizioni di campo per lattuga, pomodoro e cetriolo, con le corrispondenti maschere di annotazione impiegate per l'addestramento (ground truth manuale per la lattuga, maschere binarie del path autogenerate da SAM 3 per pomodoro e cetriolo).

Figure 5.1: Sample field images across crops and corresponding training masks

▶ **DA COMPLETARE: Origine delle immagini e condizioni di acquisizione**

Specificare la sorgente dei dati e le modalità di raccolta:

- Dispositivo di ripresa utilizzato (risoluzione e ottica), quota di acquisizione e angolazione.
- Condizioni operative nei campi (aziende agricole o serre sperimentali, condizioni meteo e fasce orarie).
- Numero complessivo di immagini raccolte per lattuga, pomodoro e cetriolo.

5.1.2 Semantic classes and labeling targets

The segmentation labels in this thesis use up to four semantic classes: background, plants, path (the traversable corridor between two crop rows), and covering material (plastic mulch film). The manual lettuce annotations use all four classes (Section 5.2.1).

The labels generated with SAM 3 (Section 5.2.2) follow three configurations, each used to train a separate network:

- **Four-class configuration:** background, plants, path, and covering material, matching the four classes of the manual lettuce annotations.
- **Three-class configuration:** background, plants, and path. No prompt targets the covering material.
- **Binary configuration:** background and path. This configuration is used both for the lettuce-only binary network and for the general dataset `Data_allPlantsSAM2`. For row following, the path mask is the input of the geometric extraction in Chapter 6.

5.2 Dataset preparation

5.2.1 Lettuce dataset with manual annotation

The manually annotated lettuce dataset, identified as `Data`, contains 762 image–mask pairs distributed across eight folders. Its masks were prepared from the manual annotations stored in `annotations.json`. The same manual reference is used for the initial architecture comparison and for evaluating the models trained with SAM 3-generated labels. These manual masks are the ground truth for all evaluations in this chapter. They contain four classes (background, plants, path, and covering material), and the path class marks only the corridor the robot should follow, not other visible paths. The three lettuce autolabel datasets (`Data_insalataSAM_4class`, `Data_insalataSAM_3class`, and `Data_insalataSAM_2class`) contain images from the same eight lettuce folders, with masks generated by SAM 3 instead of the manual annotations.

5.2.2 Automated zero-shot annotation via SAM 3

Manual segmentation requires substantial human effort, which motivated testing whether SAM 3 can replace it. SAM 3 was run offline with text prompts, and the generated masks were used without manual filtering, cleaning, or polygon correction. Keeping the imperfect masks was an intentional choice: although the manual lettuce masks were available and more accurate, the experiment measures how well a network trained on unedited autolabels performs, and therefore whether the manual annotation step can be removed. Correcting the SAM 3 masks would likely improve the trained networks, as the higher accuracy of the network trained on manual masks suggests, but it would reintroduce manual work; this trade-off was not evaluated.

All autolabel masks were generated using `SAM3SemanticPredictor` with the local `sam3.pt` checkpoint and a confidence threshold of 0.25. For each text prompt, SAM 3 returned zero or more candidate masks, and pixels with mask values

greater than 0.5 were assigned to the corresponding class. When masks of different prompts overlapped, the class was assigned according to a fixed prompt order, listed below for each dataset: later prompts were written only on pixels still labeled as background, so earlier classes were never overwritten. Pixels not assigned by any prompt remained background. The prompts and their order are:

- `Data_insalataSAM_4class`: `plants` (class 1), then `plastic mulch film` (class 3), then `central farm path` (class 2);
- `Data_insalataSAM_3class`: `plants` (class 1), then `central farm path` (class 2);
- `Data_insalataSAM_2class`: `central farm path` only (class 1);
- `Data_allPlantsSAM2`: `central path` only (class 1).

The SAM 3 masks are not compared directly with the manual annotations. Their quality enters the evaluation only through the networks trained on them (Section 5.4.1).



Figure 5.2: SAM 3 zero-shot autolabeling and mask composition workflow

5.2.3 Tomato and cucumber images

In addition to lettuce, the image collection includes tomato and cucumber crops. These images were annotated exclusively with SAM 3 and are included only in the multi-crop dataset (`Data_allPlantsSAM2`) used to train the general binary network deployed on the robot. They were not included in the training sets of the lettuce-specific models.

Because no manual ground truth was created for tomato and cucumber, these crops are not part of the quantitative evaluations in Section 5.4. Similarly, the

initial comparison among architectures (FCN, DeepLabV3, and LR-ASPP) was carried out exclusively on the lettuce dataset.

5.2.4 Dataset partitioning

The lettuce training runs use an 80% training, 10% validation, and 10% test partition. The training and validation counts are the integer parts of 80% and 10%, while the remaining samples form the test split. For the 762 image–mask pairs, this yields 609 training, 76 validation, and 77 test samples.

The test set is not used during training. Checkpoint selection relies on validation rather than test performance. The three lettuce autolabel datasets contain the same 762 images as `Data`, in the same order, and the three autolabel projects use a split file identical to that of the manual model. The 77 test images are therefore the same physical images for all four lettuce networks, and none of them belongs to the 609 training images of any of these networks. For the general binary configuration, the 77 lettuce test images were removed from `Data_allPlantsSAM2` before splitting; the remaining 1882 of 1959 image–mask pairs were split 80/10/10.

The split is random at the level of individual images: no test image is used for training, but each lettuce folder, which corresponds to one acquisition video recorded along a different path, contributes images to both the training and the test split. The test images therefore show paths whose other images were seen during training. The reported IoU values may overestimate performance on paths or fields not seen during training. This limitation applies to all networks of the comparison.

The realistic scenario uses only the test images from `insalata_grande1`, `insalata_grande2`, and `insalata_media1`: 42 of the 77 test images. These folders were recorded between parallel rows, in the area where the robot operates, from viewpoints the robot can reach. The other five folders were recorded from viewpoints the robot cannot reach and can show more than one path, while the manual masks label only the path the robot should follow. The realistic scenario is therefore a subset of the test split, not a separate test set, and its results describe only the robot's operating conditions.

5.3 Architecture comparison: FCN, DeepLabV3, and LR-ASPP

This section selects the segmentation architecture used in the rest of the thesis. FCN, DeepLabV3, and LR-ASPP were trained and compared on the manually annotated lettuce dataset.

5.3.1 Model selection and pretrained weights

FCN, DeepLabV3, and LR-ASPP were chosen from the implementations available in TorchVision. Their selection was practical, rather than the outcome of a comparison with every available architecture.

The implementation separates each network into a backbone, which extracts image features, and a segmentation head, which produces class scores at each output location. Transfer learning is implemented by loading pretrained parameters and adapting the output layers to the target labeling task, rather than initializing every network parameter from scratch.

For FCN and DeepLabV3, the selected implementations use a ResNet-50 backbone with pretrained segmentation weights. After these weights are loaded, the final convolution of the main classifier is replaced by a new 1×1 convolution with one output channel per target class; the final convolution of the auxiliary classifier is replaced in the same way.

LR-ASPP is initialized differently. The implementation uses a MobileNetV3-Large backbone and requests its default pretrained weights through the `weights_backbone` argument. However, the `weights` argument of the complete segmentation model is set to `None`. Consequently, the backbone is pretrained, whereas the LR-ASPP segmentation head is newly initialized for the requested number of classes.

This distinction is relevant to the interpretation of the model comparison. The three configurations differ not only in architecture, but also in the extent of their pretrained initialization: FCN and DeepLabV3 retain pretrained segmentation-head layers, whereas LR-ASPP starts with a new segmentation head.

5.3.2 Training configuration and loss formulation

The optimization pipeline addresses class imbalance and follows a two-stage transfer learning schedule for all evaluated architectures.

Loss function and class weighting

Agricultural scenes exhibit a strong pixel imbalance between the navigable soil corridor, the plant canopy, and background areas. To prevent the optimization from biasing toward dominant categories, the models are trained with a weighted cross-entropy loss, denoted $\mathcal{L}_{\mathrm{WCE}}$, in which the contribution of each pixel is multiplied by the weight w_c of its ground-truth class c, defined in Eq. (5.1).

Class weights are computed exclusively on the training split $\mathcal{D}_{\mathrm{train}}$ to avoid information leakage from validation and test partitions. For each class c, the total pixel count $N_c = \sum_{k \in \mathcal{D}_{\mathrm{train}}} \sum_{i,j} \mathbb{I}(Y_{i,j}^{(k)} = c)$ is accumulated across all training masks. The balanced inverse frequency inv_c is then normalized by the mean of all inverse

frequencies:

$$\text{inv}_c = \frac{\sum_{k=0}^{C-1} N_k}{C \cdot N_c}, \qquad w_c = \frac{\text{inv}_c}{\frac{1}{C}\sum_{k=0}^{C-1} \text{inv}_k}. \tag{5.1}$$

This normalization enforces $\frac{1}{C}\sum_{c=0}^{C-1} w_c = 1.0$.

For FCN and DeepLabV3, which incorporate an auxiliary classifier on the `layer3` features (output stride 8 in the dilated TorchVision backbone), the total training objective combines the primary and auxiliary losses with an auxiliary weighting factor of 0.4:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{WCE}}(\text{main}) + 0.4 \cdot \mathcal{L}_{\text{WCE}}(\text{aux}). \tag{5.2}$$

For LR-ASPP, which lacks an auxiliary head, training relies solely on the primary loss $\mathcal{L}_{\text{WCE}}(\text{main})$.

Data augmentation and two-stage optimization

Data augmentation is applied only to the training subset (`full_train_dataset`): random horizontal flips with probability 0.5, random rotations between $-10°$ and $10°$, and, with probability 0.8, a color jitter of brightness and contrast (factors in $[0.8, 1.2]$) and saturation (factors in $[0.85, 1.15]$) to handle variable outdoor illumination. The validation and test sets (`full_eval_dataset`) are not augmented. Images are not resized, and all images are normalized with the ImageNet mean and standard deviation.

The networks were trained on an NVIDIA GeForce RTX 2080 SUPER with 8 GB of memory, with a batch size of 2.

Optimization proceeds across 200 epochs in two distinct phases:

1. **Stage 1 (Warmup, Epochs 1–10):** Backbone parameters are frozen by setting `requires_grad = False`. Only the segmentation heads (and auxiliary classifiers when present) are optimized using AdamW with an initial learning rate of 10^{-3} and weight decay of 10^{-4}. A cosine-annealing scheduler with $T_{\max} = 200$ and $\eta_{\min} = 10^{-6}$ updates the learning rate after each epoch. Because the newly initialized output layers make large errors in the first epochs, their large error gradients could damage the pretrained weights; freezing the reused layers during these epochs gives the new layers time to learn reasonable weights [16, Chapter 11].

2. **Stage 2 (Differential Fine-Tuning, Epochs 11–200):** At epoch 10, the backbone is unfrozen. A new AdamW optimizer is instantiated with decoupled parameter groups: the backbone is assigned a learning rate of 10^{-5} and the segmentation heads a learning rate of 10^{-4}. The lower learning rate of the unfrozen backbone limits changes to its pretrained weights [16, Chapter 11].

The cosine-annealing scheduler is re-initialized across the remaining 190 epochs ($T_{\max} = 190, \eta_{\min} = 10^{-6}$).

Early stopping was not used: all three networks were trained for the full 200 epochs, so that the complete training and validation loss curves could be recorded. Model checkpoints are evaluated after each epoch on the unaugmented validation set; `best.pth` is saved whenever the validation loss decreases below its historical minimum, and `last.pth` records the final epoch state.

[SPAZIO FIGURA / SCHEMA]

Training and validation loss across epochs for FCN, DeepLabV3, and LR-ASPP

Curve di apprendimento durante le 200 epoche: loss di training e loss di validazione (dai file `training_log.csv`) per le tre architetture candidate sul dataset di lattuga.

Figure 5.3: Training and validation loss across epochs for FCN, DeepLabV3, and LR-ASPP

5.3.3 Evaluation metrics and results

The comparison uses the three `*_validation_metrics.csv` files produced by the validation script, one per network. Each contains the aggregate metrics, the number of parameters, the checkpoint epoch, and the IoU values for background, plants, soil, and covering material. The reported mIoU agrees with the arithmetic mean of these four class IoUs, including background.

Table 5.1 reports the exported metrics. Inference times have been converted from seconds to milliseconds. The FPS values equal the reciprocal of the mean recorded time; they are not measurements of the complete robot processing or control rate. Checkpoint epochs identify the evaluated weights, not training duration.

FCN and DeepLabV3 achieve mIoU values of 0.9232 and 0.9228, respectively, compared with 0.9108 for LR-ASPP. Using the unrounded CSV values, the LR-ASPP deficit is 1.24 percentage points relative to FCN and 1.19 relative to DeepLabV3. Its mean recorded time is 10.41 ms, compared with 58.24 ms and 86.87 ms. These timings correspond to a 5.60-fold reduction relative to FCN and an 8.35-fold reduction relative to DeepLabV3.

Table 5.2 provides the class-level comparison. LR-ASPP has lower IoU than both alternatives in each class, although its soil/path IoU remains 0.9546. The data

Table 5.1: Architecture comparison from the manual lettuce validation exports. Timing values are those recorded in the CSV files, not end-to-end robot latency. Parameter counts refer to the trained four-class models and include the auxiliary classifier of FCN and DeepLabV3, which is used only during training.

Model	Parameters (M)	Epoch	mIoU	Path IoU	Mean time (ms)	FPS
FCN	35.3	38	0.9232	0.9621	58.24	17.17
DeepLabV3	42.0	43	0.9228	0.9651	86.87	11.51
LR-ASPP	3.2	194	0.9108	0.9546	10.41	96.07

support a trade-off between accuracy and latency, not a claim of equal segmentation accuracy.

Table 5.2: Per-class IoU from the same manual lettuce validation exports. Plants, soil, and covering material correspond to `pianta`, `suolo`, and `copertura` in these files.

Model	Background	Plants	Soil/path	Covering material
FCN	0.9464	0.9280	0.9621	0.8565
DeepLabV3	0.9464	0.9277	0.9651	0.8519
LR-ASPP	0.9394	0.9120	0.9546	0.8372

5.3.4 Selection of LR-ASPP

LR-ASPP was adopted because the substantial reduction in recorded inference time justified accepting an mIoU approximately 1.2 percentage points below the other two configurations. It was not the most accurate model, but offered the practical accuracy–latency compromise sought for robot perception. FCN and DeepLabV3 had higher IoU but substantially longer recorded inference times. The subsequent SAM-supervised training runs therefore used LR-ASPP while varying the output label set.

Two limitations apply to this comparison. First, the metrics are computed on the validation split, which is also used to select `best.pth` through the minimum validation loss, so they are optimistic for all three networks. Second, the three times were measured with the same validation script, one image at a time at its original resolution, with GPU synchronization before and after the forward pass and the argmax, but in separate runs whose hardware load was not recorded. Recorded times depend on these conditions: in the evaluation scripts of Section 5.4, the same LR-ASPP checkpoint records mean times between 0.010 s and 3.67 s per image. The 5.60-fold and 8.35-fold ratios therefore hold only if the three runs had comparable conditions.

This choice is based on the available offline comparison. The records do not establish a hardware-independent speed ranking or demonstrate that the integrated robot meets its control timing requirements. Those requirements must be assessed using the deployed model and the complete perception and control pipeline.

5.4 Supervision strategy comparison

5.4.1 Training configurations and evaluation targets

Table 5.3 summarizes the five LR-ASPP networks considered in this comparison. This study is separate from the initial comparison of the three candidate architectures. The first four networks are trained on lettuce images only, so that their predictions can be evaluated against the manual ground truth of the same crop. The fifth network reproduces the general binary configuration deployed on the robot, trained on the complete SAM 3 collection without the 77 lettuce test images.

Table 5.3: LR-ASPP training configurations in the supervision comparison. Class counts include background; all networks are evaluated against the manual lettuce ground truth. The checkpoint epoch is the epoch of the evaluated `best.pth`, selected by minimum validation loss.

Configuration	Training data (labels)	Output classes	Classes	Epoch
Manual baseline	`Data` (manual)	Background, plants, path, covering material	4	194
SAM 3 four-class	`Data_insalataSAM_4class` (SAM 3)	Background, plants, path, covering material	4	30
SAM 3 three-class	`Data_insalataSAM_3class` (SAM 3)	Background, plants, path	3	38
SAM 3 binary	`Data_insalataSAM_2class` (SAM 3)	Background, path	2	18
SAM 3 binary, general	`Data_allPlantsSAM2` without the 77 lettuce test images (SAM 3)	Background, path	2	27

Manual masks from `Data` supply the ground truth for all comparative evaluations. The evaluation measures the generalization capability of the trained networks rather than the static quality of the raw SAM 3 masks. Specifically, no direct pixel-level comparison is performed between uncurated SAM 3 masks and manual annotations; instead, each trained LR-ASPP network executes forward inference in evaluation mode (`eval()`), computing the discrete predicted class map via:

$$\hat{Y}_{i,j} = \arg \max_{c \in \{0,\ldots,C-1\}} P(Y_{i,j} = c \mid I), \tag{5.3}$$

which is subsequently compared against the human ground-truth mask.

Evaluation uses the Intersection over Union (IoU) metric:

$$\mathrm{IoU}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \tag{5.4}$$

where TP_c, FP_c, and FN_c denote true positives, false positives, and false negatives for class c, respectively. Mean IoU (mIoU) denotes the arithmetic mean across all evaluated classes, including background where applicable.

Performance is evaluated on two subsets of the 77-image test partition:

- **Complete test set (77 images):** Spans all eight lettuce acquisition folders (`insalata_grande1`, `insalata_grande2`, `insalata_media1`, `insalata_media2`, `insalata_media3`, `insalata_piccola`, `insalata_piccola_1`, and `insalata_piccola_2`), including viewpoints that the robot cannot reach.
- **Realistic operational scenario (42 images):** Restricts evaluation to `insalata_grande1`, `insalata_grande2`, and `insalata_media1`, described in Section 5.2.4. The networks are not retrained on this subset.

5.4.2 Benchmark results across supervision strategies

All results in this section are computed on the test split. The four comparisons evaluated in `compareInsalata` are reported separately, each on the complete test set and on the realistic scenario.

Three-class autolabel model: salad and path

This comparison evaluates the two foreground categories shared by the 4-class manual baseline and the 3-class autolabel model (`InsalataSAM_3class`); the other classes are ignored. As reported in Table 5.4, the manual model achieves a mean IoU of 0.9406 on the complete test set and 0.9544 in the realistic scenario. The 3-class autolabel model reaches a path IoU of only 0.4124 on the complete test set, and 0.8966 (salad IoU 0.8779) in the realistic scenario.

Four-class autolabel model

Table 5.5 presents the complete 4-class evaluation comparing the manual baseline with the 4-class autolabel network (`InsalataSAM_4class`). The manual model reaches an mIoU of 0.9124 on the complete test set and 0.9108 in the realistic scenario; the 4-class autolabel network reaches 0.4639 and 0.5925. In the realistic scenario the deficit is concentrated in plastic mulch film (`copertura` IoU 0.2495) and background (IoU 0.3449), while its path IoU is 0.8997. Visual inspection of

Table 5.4: Test set performance of the three-class autolabel model on salad and path (records from `3_classi` and `3_classi_realisticScenario`). Mean IoU denotes the arithmetic average of salad and path IoU.

Scenario	Model	Salad IoU	Path IoU	Mean IoU
Complete test set	Manual baseline (4-class)	0.9246	0.9566	0.9406
	Autolabel SAM (3-class)	0.8130	0.4124	0.6127
Realistic scenario	Manual baseline (4-class)	0.9473	0.9615	0.9544
	Autolabel SAM (3-class)	0.8779	0.8966	0.8873

the SAM 3 masks showed that the `"plastic mulch film"` prompt segmented the covering poorly and also assigned the greenhouse plastic cover to this class. Since the greenhouse cover is background in the manual masks, this is consistent with the low IoU of both covering and background. A further plausible cause is that the prompt produces inaccurate masks where the film is soiled or damaged. These explanations come from visual inspection; the SAM 3 masks were not compared quantitatively with the manual annotations.

Table 5.5: Complete four-class test set results (records from `4_classi` and `4_class_realisticScenario`). mIoU is the average of all four classes including background.

Scenario	Model	Background	Salad	Path	Mulch film	mIoU
Complete test	Manual baseline	0.9426	0.9246	0.9566	0.8257	0.9124
	Autolabel SAM	0.4631	0.8081	0.4124	0.1719	0.4639
Realistic test	Manual baseline	0.9188	0.9473	0.9615	0.8157	0.9108
	Autolabel SAM	0.3449	0.8757	0.8997	0.2495	0.5925

Binary autolabel model trained on lettuce only

Table 5.6 compares path segmentation between the manual baseline and the binary autolabel network trained only on lettuce (`InsalataSAM_2class`, records from `2_classi` and `2_classi_realisticScenario`). Its labels come from the same `central farm path` prompt used for the path class of the 3-class and 4-class datasets. The network reaches a path IoU of 0.3917 on the complete test set and 0.9013 in the realistic scenario.

Table 5.6: Path IoU of the binary autolabel model trained on lettuce only, on the test set. The manual model path is class 2; the binary model path is class 1.

Scenario	Model	Path IoU
Complete test set	Manual baseline (4-class)	0.9566
	Autolabel SAM (2-class, lettuce only)	0.3917
Realistic scenario	Manual baseline (4-class)	0.9615
	Autolabel SAM (2-class, lettuce only)	0.9013

General binary autolabel model

Table 5.7 reports the binary network trained on the general SAM 3 collection without the lettuce test images (project `allPlantsSAM_test`, records from `2_class_allPlants_clean` and `2_class_allPlants_clean_realisticScenario`). Its labels come from the `central path` prompt, and its training data contain tomato and cucumber images together with the remaining lettuce frames. The evaluated checkpoint is the one with the lowest validation loss (epoch 27, loss 0.1159). The network reaches a path IoU of 0.6369 on the complete test set and 0.9071 in the realistic scenario.

Table 5.7: Path IoU of the general binary autolabel model, trained without the lettuce test images, on the test set. The manual model path is class 2; the binary model path is class 1.

Scenario	Model	Path IoU
Complete test set	Manual baseline (4-class)	0.9566
	Autolabel SAM (2-class, general)	0.6369
Realistic scenario	Manual baseline (4-class)	0.9615
	Autolabel SAM (2-class, general)	0.9071

5.4.3 Path segmentation with autolabel supervision

Table 5.8 collects the path IoU of every network from Tables 5.4–5.7.

Table 5.8: Path IoU on the test set for all networks of the supervision comparison.

Network	Complete test set	Realistic scenario
Manual baseline (4-class)	0.9566	0.9615
SAM 3 four-class (lettuce)	0.4124	0.8997
SAM 3 three-class (lettuce)	0.4124	0.8966
SAM 3 binary (lettuce)	0.3917	0.9013
SAM 3 binary (general, test images excluded)	0.6369	0.9071

The results support the following conclusions:

1. **The manual model is the best overall.** It has the highest IoU in every comparison, with path IoU 0.9566 on the complete test set and 0.9615 in the realistic scenario.

2. **Lettuce-only autolabel training.** In the realistic scenario, the binary network trained only on lettuce reaches a path IoU of 0.9013, 6.0 percentage points below the manual model. The three lettuce autolabel networks differ by less than 0.5 percentage points on path IoU in this scenario (0.8966–0.9013), and on the complete test set the binary network is the lowest of the three (0.3917 against 0.4124). These data do not show that removing the plant and covering classes improves path segmentation.

3. **General autolabel data.** The general binary network reaches a path IoU of 0.9071 in the realistic scenario, close to the lettuce binary network (0.9013), and 0.6369 on the complete test set, against 0.3917. Its training data differ in several ways at once (tomato and cucumber images, a different set of lettuce frames, and the `central path` prompt), so the improvement cannot be attributed to one factor.

4. **Complete test set versus realistic scenario.** Both scenarios evaluate the same networks, trained on all eight lettuce folders; the realistic scenario is only a subset of the test images. All autolabel networks lose much of their path accuracy on the complete test set (path IoU 0.39–0.41 for the lettuce networks and 0.64 for the general network), while the manual model does not (0.9566). The drop comes from the five folders outside the operational scenario. Their images were taken from viewpoints the robot cannot reach and can show several paths: the manual masks label only the path the robot should follow, whereas the autolabel networks segment every visible path, and the IoU counts the additional paths as false positives. In addition, visual inspection showed that SAM 3 produced less accurate path masks in images taken from unusual viewpoints or where the path between the rows is not clearly visible, so the networks were trained on noisier labels for these conditions. The complete-test values therefore combine segmentation errors, label noise, and this difference in label definition; the three effects were not separated quantitatively.

5. **Full semantic segmentation.** With an mIoU of 0.5925 in the realistic scenario, the 4-class autolabel network does not support replacing manual annotation when all four classes are required.

6. **Sufficiency for navigation.** Path IoU values of 0.9013 (lettuce binary) and 0.9071 (general binary) in the realistic scenario imply that more than 90% of the ground-truth path pixels are predicted as path, since recall $\mathrm{TP}/(\mathrm{TP} + \mathrm{FN})$ is never lower than IoU. Whether this accuracy is sufficient for row following depends on how the residual errors are distributed along the corridor boundaries. It must be established through an application threshold or a downstream test of the path extraction and controller (Chapters 6 and 8), not from IoU alone.

5.4.4 Network deployed on the robot

After the lettuce study, the binary labeling was extended to the complete image collection, labeled entirely by SAM 3 with the `central path` prompt and without manual correction (`Data_allPlantsSAM2`). The binary LR-ASPP network trained on this collection (checkpoint `best_allsam2.pth`, epoch 25) is the network used on the robot in Chapters 7 and 8. Its training data include the lettuce test images, so its own IoU on them would not be an independent measure and is not reported. The general network of Table 5.7, trained with the same labels on the collection without those images, gives the independent estimate for this configuration: path IoU 0.9071 in the realistic scenario and 0.6369 on the complete test set. It is a separate training run (best epoch 27 instead of 25), so the estimate refers to the configuration rather than to the exact deployed weights. The deployed network was also assessed qualitatively, by visual inspection of its predictions over the dataset, and then in simulation and in field tests (Chapter 8). The visual inspection also showed that it segments adjacent or non-traversable corridors, which the manual lettuce masks leave as background. The checkpoint documentation identifies `Data_allPlantsSAM2` as its training set, whereas the historical training script refers to `Data_allPlantsMIX`; the checkpoint does not store the dataset path.

For integration with the geometric extraction, a binary network outputs the path class directly. The extractor `pf4.py` receives the predicted class map and a `path_class` parameter (default 1) and binarizes the map internally, so a multi-class output could also be used by setting `path_class` to the index of its path class. It uses only the path class: it analyzes 24 horizontal strips and fits the left and right corridor edges with iteratively reweighted least squares (Chapter 6).

Chapter 6

Geometric Extraction of the Local Navigation Reference

This chapter describes the geometric extractor implemented in `pf4.py` (referred to as Method 4 in the experimental evaluation). It converts a segmented corridor mask into a local navigation reference by extracting intervals in horizontal strips, associating their left and right boundaries from bottom to top, and fitting each edge independently with an iteratively reweighted linear fit. The centerline reference is the analytic mean of the two fitted edge lines.

6.1 Input representation and edge-line models

The function expects a two-dimensional label mask $M(y, x)$ of height H and width W. Image coordinates increase to the right in x and downward in y. The argument `path_class` identifies the class from which corridor intervals are extracted; its default is 1. A binary input can therefore use 1 for the target region and 0 elsewhere. For a multiclass segmentation mask, this argument must match the actual label encoding, or the mask must first be converted appropriately. The extractor does not infer the class mapping or use a separate plant class. A target-region label is not, by itself, a guarantee of traversable space.

The left and right boundaries are represented by

$$x_L(y) = m_L y + q_L, \qquad x_R(y) = m_R y + q_R. \tag{6.1}$$

Treating x as a function of y permits vertical image lines without an infinite slope. The center reference is

$$x_C(y) = \frac{x_L(y) + x_R(y)}{2} = m_C y + q_C, \qquad m_C = \frac{m_L + m_R}{2}, \quad q_C = \frac{q_L + q_R}{2}. \tag{6.2}$$

The extractor therefore assumes locally straight image-space boundaries. The models do not reconstruct terrain or estimate a metric trajectory. Their residuals and width estimates are in pixels, not lateral distances in meters. The two edges need not be parallel: their separation can change linearly with image height.

At each image height, the reference lies halfway between the two predicted boundaries. This construction is not generally equivalent to fitting the observed interval midpoints: the two edge fits can exclude different border-touching samples and assign different residual weights. Keeping the boundaries separate allows each visible side to contribute its own observations, although both sides must have enough samples.

The processing stages are:

1. select a lower region of interest and clean the target-class mask;
2. extract and merge projected intervals in horizontal strips;
3. select a seed interval by edge support;
4. associate intervals upward through the image;
5. fit the non-clipped left and right boundaries independently;
6. validate the edge geometry, calculate quality indicators, and rasterize the center reference.

6.2 Region of interest and mask preparation

The ROI begins at $y_0 = \lfloor \rho H \rfloor$ and extends to the bottom of the image. The default starting ratio is $\rho = 0.30$ for Method 4. The binary target mask is

$$B(y, x) = \mathbf{1}\{M(y_0 + y, x) = \texttt{path_class}\}, \qquad 0 \le y < H_R = H - y_0. \tag{6.3}$$

Opening with a 5×5 elliptical kernel is applied first, then closing with a 9×9 rectangular kernel. Connected components with fewer than 500 pixels are removed; all sufficiently large components are retained, not only the largest one.

Let $\widetilde{B}$ denote the cleaned binary ROI mask. It supplies both interval extraction and local support calculations. These support calculations do not use the original multiclass mask to compute separate soil and plant ratios.

Morphology and interval merging can bridge interruptions or change region connectivity. Consequently, support in $\widetilde{B}$ measures agreement with the processed mask, not independently verified scene geometry. Kernel sizes, component areas, and distance thresholds are in input-mask pixels and depend on image resolution and camera configuration.

6.3 Horizontal strips and candidate intervals

6.3.1 Strip construction and column projection

The extractor uses $N = 24$ strips by default, indexed from bottom to top. Their height is $h = \max(1, \lfloor H_R/N \rfloor)$. For strip index s, the ROI-relative bounds are

$$y_s^- = \max(0, H_R - (s+1)h), \qquad y_s^+ = \min(H_R, H_R - sh), \tag{6.4}$$

with the lower image row bound y_s^+ excluded. Empty strips are skipped. The representative full-image coordinate is $y_s = y_0 + (y_s^- + y_s^+)/2$, following the code's bound-based convention. Integer division may leave a remainder above the constructed strips.

For example, with $H_R = 157$ and $N = 24$, the strips cover the lowest $24 \times 6 = 144$ ROI rows. The upper 13 ROI rows do not supply candidates; the ROI and the region actually sampled by the strips are therefore not identical.

Within each strip, the cleaned mask is summed vertically. A column is active whenever its sum exceeds zero, so even one target pixel activates that column. Contiguous active columns form raw intervals. Neighboring intervals are merged when the number of inactive columns between them is at most `max_interval_gap`, whose default is 30 pixels. For a merged interval $[\ell_i, r_i]$, the width is

$$d_i = r_i - \ell_i + 1. \tag{6.5}$$

Intervals narrower than 8 pixels are discarded. There is no fixed maximum interval width. Instead, an interval touching both image margins is rejected if the mean of the entire cleaned binary strip is below 0.70. With `border_margin`=2, the flags are

$$t_{L,i} = \mathbf{1}\{\ell_i \le 2\}, \qquad t_{R,i} = \mathbf{1}\{r_i \ge W - 1 - 2\}. \tag{6.6}$$

These flags indicate possible truncation by the image boundary, rather than verified physical corridor edges.

Column projection merges evidence across different vertical locations; a merged interval can also contain horizontal gaps. Neither its width nor its endpoints prove that its rectangular interior is entirely target class. This matters when interpreting intervals around occlusions or fragmented segmentation.

6.3.2 Local support around interval boundaries

Method 4 computes $u_{L,i}$ and $u_{R,i}$ in separate bands centered on ℓ_i and r_i. The effective half-width is $\max(2, \texttt{local_band_half_width})$, equal to 5 by default. These means quantify target-pixel occupancy around each boundary; they are not boundary-detection probabilities. Because a band straddles a boundary, it need not be entirely target class even for a clean interval.



Figure 6.1: Horizontal strip decomposition and column-wise projection for path interval detection

Edge support ranks seed candidates and contributes to the final quality score. It is not an acceptance threshold and does not weight the line fit. In particular, `min_edge_support`=0.20 is present in the function signature but unused in the active implementation.

6.4 Initialization and bottom-up association

6.4.1 Edge-support seed selection and gap-limited association

Method 4 pools all candidates in the first six constructed strips from the bottom and selects the seed by maximizing $\min(u_{L,i}, u_{R,i})$, then interval width. No minimum-support threshold is applied, so even a weakly supported candidate can initialize tracking.

For a candidate interval $[\ell_i, r_i]$ and the previously accepted interval $[\ell_{\text{prev}}, r_{\text{prev}}]$ of width d_{prev}, the boundary displacement is $\delta_i = \max(|\ell_i - \ell_{\text{prev}}|, |r_i - r_{\text{prev}}|)$. A candidate is admissible if $\delta_i \leq D_4$, and the admissible candidate with the lowest cost $J_{4,i}$ is accepted:

$$D_4 = \max(28, 0.5 d_{\text{prev}}), \qquad J_{4,i} = \delta_i + 0.4|d_i - d_{\text{prev}}|. \tag{6.7}$$

Edge support does not enter this cost or an additional tracking gate.

A counter records consecutive strips without a compatible candidate and resets after each successful association. With `max_gap_strips`=3, tracking stops on the fourth consecutive miss. Stopping does not directly return `lost`: the accumulated points proceed to fitting if at least three are available. The maximum miss count, including the miss that triggers stopping, enters the quality score. The previous interval remains unchanged during gaps.

The stopping rule and gap penalty have different effects: three misses are tolerated for continued association, but already reduce the gap score to zero. A fourth miss stops the search without automatically rejecting a reference supported by the observations collected earlier.

Method 4 performs greedy spatial association within one frame, not temporal tracking. Each accepted interval contributes two boundary observations, and at least three accepted intervals are required before the edge-fitting stage.

6.5 Separate reweighted fitting of the corridor edges

6.5.1 Excluding image-boundary observations

The left fit uses only accepted intervals not flagged as touching the left image margin. The right fit independently excludes intervals flagged as touching the right margin. At least two observations per edge must remain. The two fits can therefore use different strips and span different vertical ranges. A clipped boundary is not replaced by a one-sided centerline estimate: if either edge lacks sufficient samples, the function returns an empty output with `lost` status.

The retained samples are interval endpoints from the cleaned column projection. They are not points obtained by a dedicated edge detector. Excluding border-touching samples reduces the influence of image truncation, but does not establish the correctness of the remaining boundaries.

6.5.2 Residual-based reweighting

Each edge fit starts with uniform weights. There is no semantic, interval-width, or vertical-position base weighting. For an edge sample (x_i, y_i), four reweighting iterations are performed by default, with scale $\sigma = 18$ pixels:

$$e_i^{(t)} = x_i - m^{(t)} y_i - q^{(t)}, \qquad w_i^{(t+1)} = \frac{1}{1 + (e_i^{(t)}/\sigma)^2}. \tag{6.8}$$

Each iteration calls `numpy.polyfit` with degree one and updated weights. Its weights multiply unsquared residuals [21]; the fitted objective at that iteration is

$$(m, q) = \arg\min_{m,q} \sum_i \left[w_i (x_i - m y_i - q) \right]^2 . \tag{6.9}$$

Thus, the effective coefficient of each squared residual is w_i^2, not w_i. The implementation should not be interpreted as a different standard robust-loss estimator solely from the form of its weight rule. Weights are recomputed from the current model rather than accumulated across iterations; no convergence-based stopping criterion is used.

The final residual for each retained sample is its absolute horizontal deviation from the corresponding fitted edge. Define $\bar{e}$ as the mean over the concatenated left and right residual arrays, and

$$F = \mathrm{clip}(1 - \bar{e}/\sigma, 0, 1). \tag{6.10}$$

Edges with more retained samples contribute more terms to this mean. Reweighting reduces the influence of inconsistent points but cannot correct an erroneous

[SPAZIO FIGURA / SCHEMA]

Reweighted line fitting of corridor boundaries and centerline construction

Diagramma del processo di fitting robusto separato dei bordi sinistro (x_L) e destro (x_R) tramite re-weighting iterativo dei residui con fattore lorentziano ($\sigma = 18$ px) e calcolo dell'asse centrale di guida $x_C(y)$.

Figure 6.2: Reweighted line fitting of corridor boundaries and centerline construction

association chain that supports a coherent wrong line. It does not impose parallel edges or estimate genuine path curvature.

6.6 Quality indicators, validation, and image-space output

6.6.1 Method 4: geometric checks and a composite score

Method 4 validates the fitted boundaries before constructing the output. The modeled corridor width is

$$D(y) = x_R(y) - x_L(y) = \Delta m\, y + \Delta q, \qquad \Delta m = m_R - m_L, \quad \Delta q = q_R - q_L. \tag{6.11}$$

Rows with $D(y) \geq d_{\min}$, where $d_{\min} = 8$ px is the minimum interval width, are treated as valid. Let $y_{\text{top}}^{\text{obs}}$ and $y_{\text{bot}}^{\text{obs}}$ be the smallest and largest heights of the retained edge samples. If $|\Delta m| < 10^{-6}$, the edges are treated as parallel and the check requires $\Delta q \geq d_{\min}$. Otherwise the width reaches $d_{\min}$ at $y_\times = (d_{\min} - \Delta q)/\Delta m$, and the valid range is

$$y_v^{\text{top}} = \begin{cases} \max(y_{\text{top}}^{\text{obs}}, y_\times) & \Delta m > 0 \\ y_{\text{top}}^{\text{obs}} & \Delta m < 0 \end{cases}, \qquad y_v^{\text{bot}} = \begin{cases} H - 1 & \Delta m > 0 \\ \min(H-1, y_\times) & \Delta m < 0 \end{cases}. \tag{6.12}$$

The fit is rejected if the crossing lies inside the observed range, $y_{\text{top}}^{\text{obs}} + 0.05H < y_\times < y_{\text{bot}}^{\text{obs}}$, because this indicates swapped edges or an association across different regions rather than a corridor that ends. It is also rejected if the valid range is shorter than $0.10(H - y_0)$. Other crossing positions do not reject the fit directly: they shorten the valid range, and the length condition then decides. In particular, when $\Delta m > 0$ and the crossing lies below the observed range, the modeled width stays below $d_{\min}$ over all observed samples, yet the fit is accepted if the range from $y_\times$ to the image bottom satisfies the length condition; the reference is then rendered only below $y_\times$. The median of $D(y)$ at 16 evenly spaced heights in the valid range (in the observed range if the check fails) is returned as `width`.

A separate indicator uses all accepted intervals, including those with border-touching endpoints. For K intervals in association order, let $z_j = r_j - \ell_j$; unlike extraction width d_j, this quantity omits the inclusive-endpoint +1. The width-irregularity indicator is

$$E_D = \frac{Q_{0.90}\left(\{|z_{j+2} - 2z_{j+1} + z_j|\}_{j=1}^{K-2}\right)}{\max(\text{median}_j\, |z_j|, 1)}, \qquad A = \text{clip}(1 - E_D/0.30, 0, 1), \tag{6.13}$$

where $Q_{0.90}$ is the 90th percentile. These are second differences in sample order, not derivatives normalized by vertical distance. If strips are skipped, sample spacing

is unequal; even a width varying linearly with image height need not yield zero second differences. The indicator therefore measures an implementation-specific irregularity, not a perspective-invariant physical curvature.

The tracking fraction T is the number of accepted intervals divided by the number of strips whose center lies at or below y_v^{top}; if the geometric check fails, all constructed strips are counted. Let S be the mean of the left support values used by the left fit and the right support values used by the right fit, pooled together. With maximum consecutive miss count $g_{\max}$ and allowed gap $g_a = 3$,

$$G = \text{clip}(1-g_{\max}/\max(g_a,1),0,1), \qquad C_4 = 0.25T+0.20S+0.25F+0.20A+0.10G. \tag{6.14}$$

The result is `lost` if the geometric check fails, $\bar{e} > 2\sigma$, or $C_4 < 0.38$. Otherwise, it is `degraded` for $C_4 < 0.60$ and `ok` for $C_4 \geq 0.60$.

A result marked `lost` returns a zero output matrix before rasterization, with no centerline model in the post-fit rejection branch. For an accepted result, the mean edge line is rasterized from the lower (larger y) of the top of the highest accepted strip and y_v^{top} down to y_v^{bot}, which is the bottom of the image unless the edges converge downward ($\Delta m < 0$), and then dilated with a 3×3 elliptical kernel. The line can therefore extrapolate below the seed. Because all rendered rows lie in the valid range, the modeled width there is at least $d_{\min}$, and the reference is never drawn beyond a crossing of the two edges.

Diagnostics report the edge and centerline models, mean residual, support, tracking fraction, median fitted width, width irregularity, maximum gap, row bounds, and cleaned ROI mask. Earlier failures return `lost` with reason strings such as `no_seed_candidate`, `tracking_failed`, or `insufficient_edge_points`.

6.6.2 Interpretation of status and confidence

The confidence score C_4 is a heuristic combining processed-mask edge support with geometric and gap indicators. It is not a calibrated probability. A downstream controller must inspect the returned status; the function does not specify a robot stopping or recovery policy. Malformed inputs, unsuitable parameter values, or numerical exceptions are not comprehensively handled by the explicit failure branches.

6.7 Default configuration

Table 6.1 lists the default parameter values for Method 4. These are the defaults of `pf4.py`, not a statement that a controller uses them or that they are experimentally optimal. Overrides and mask encodings must be recorded for each experiment.

Table 6.1: Default configuration of Method 4 (`pf4.py`). Distances and areas are expressed in input-mask pixels.

Setting	Value
Target-class identifier	1
ROI starting ratio ρ	0.30
Number of strips N	24
Opening / closing kernels	5×5 / 9×9
Minimum component area	500 px
Minimum interval width	8 px
Maximum merged gap	30 px
Image-border margin	2 px
Full-strip occupancy gate	0.70
Local support-band half-width	5 px
Seed search window	6 strips
Jump-limit lower bound	28 px
Width-change cost coefficient	0.4
Consecutive misses allowed	3
Residual scale σ / iterations	18 px / 4
Residual rejection threshold	$2\sigma = 36$ px
Crossing tolerance above the observed range	$0.05H$
Minimum valid reference length	$0.10(H - y_0)$
Width-error score scale	0.30
Lost / acceptable score thresholds	0.38 / 0.60
Output dilation kernel	3×3

The two-line representation retains boundary information separately, but requires sufficient non-clipped samples on both sides. Independent edge fits may be supported over different height ranges, and their mean can include extrapolation of one or both boundaries. A wide or well-supported interval is not necessarily the intended row.

6.8 Effect of strip height and number

The strip count controls vertical sampling and the maximum number of interval observations. More strips provide thinner vertical regions and potentially more points per boundary fit. Taller strips aggregate more mask pixels but mix evidence from different image heights, and column projection can conceal interruptions. Increased strip height does not guarantee more reliable boundary estimates or a numerically better fit.

For a 224×224 input with $\rho = 0.30$, Method 4 has $y_0 = 67$ and $H_R = 157$. Table 6.2 shows strip heights for different strip counts using these dimensions. At $N = 24$, the strip height is 6 pixels. A fit can use fewer than N points because of seed location, missed strips, early stopping, or border-point exclusion; the two edges can have different sample counts.

Table 6.2: Strip heights for an illustrative 224×224 mask with $\rho = 0.30$ ($H_R = 157$). The point count is an upper bound per edge, not a guaranteed number of fitted samples.

Strips N	Strip height (px)	Max. edge samples
24	6	24
20	7	20
16	9	16
12	13	12
8	19	8

Changing N also changes the vertical extent of a fixed six-strip seed window and the extent of a three-strip tolerated gap. These settings should therefore be considered together. The width-irregularity indicator depends on the sampling sequence, while the tracking fraction uses as its denominator the strips in the valid range (all constructed strips when the geometric check fails). Selecting N requires evaluating boundary and centerline errors, accepted-point counts, rejection rates, and processing time under the intended image resolution and field conditions. No optimal strip count is inferred from the implementation alone.

6.9 Baseline algorithms for comparative evaluation

To quantify the performance of decoupled dual-edge regression against alternative geometric strategies, three baseline extractors are integrated into the evaluation pipeline: a center-point strip tracker, a column-histogram extractor, and a RANSAC-based dual-edge extractor.

6.9.1 Center-point strip tracking baseline

The center-point strip tracker (`pf_centri.py`) shares the pre-processing stage, region of interest ($y_0 = \lfloor 0.30H \rfloor$), and horizontal strip decomposition ($N = 24$) of Method 4. In each strip, horizontal intervals are extracted from the binary corridor mask. Instead of retaining the left and right boundary coordinates independently, the algorithm computes the midpoint and width of each candidate interval:

$$x_C = \frac{x_L + x_R}{2}, \qquad w = x_R - x_L + 1. \tag{6.15}$$

The seed is the interval in the bottom six strips with the largest number of target pixels (the sum of its column projection), with ties broken by width. Successive intervals are associated bottom-up by minimizing a transition cost combining lateral center displacement and corridor width variation:

$$J_{\text{center}} = |x_C^{(k+1)} - x_C^{(k)}| + 0.4\,|w^{(k+1)} - w^{(k)}|, \tag{6.16}$$

subject to a maximum allowable jump $|x_C^{(k+1)} - x_C^{(k)}| \leq \max(28, 0.5\,w^{(k)})$.

As in Method 4, association stops after the fourth consecutive strip without a compatible candidate, and at least three accepted intervals are required. A single centerline model $x_C(y) = m_C y + q_C$ is then fitted to the collected centers $(y_k, x_{C,k})$ with four reweighting iterations of the rule in Eq. (6.8) ($\sigma = 18$ px). Two details differ from Method 4: the initial weights are proportional to the pixel counts of the accepted intervals (normalized to mean one), and each new weight factor multiplies the previous weights, so down-weighting accumulates across iterations. Border-touching intervals are not excluded. The result is `lost` if the mean residual exceeds 2σ or if the score $0.30T + 0.35F + 0.25A + 0.10G$ (with T computed over all constructed strips) is below 0.38, where A is computed from the interval widths d; accepted lines are rendered from the top of the highest accepted strip to the bottom of the image. Because interval midpoints are formed prior to fitting, a disturbance on one border shifts $x_{C,k}$ directly and enters the fitted centerline.

6.9.2 Column-histogram baseline

The column-histogram baseline (`pf_histogram.py`) is a simplified column-projection method related to SegMin [3]. SegMin sums a plant segmentation mask, accumulated over recent frames and refined with depth, along the image columns, smooths the profile with a moving average, and selects the global minimum [3]. `pf_histogram` instead counts target-class (path) pixels in each column, without temporal accumulation, depth, smoothing, morphological cleaning, or strip subdivision:

$$h(x) = \sum_{y=y_0}^{H-1} \mathbf{1}\{M(y,x) = \texttt{path_class}\}, \qquad x \in [0, W-1]. \tag{6.17}$$

Its default starting ratio is $\rho = 0$, so the whole image is used. If no column contains target pixels, the result is `lost`. Otherwise, the target column x_{target} is the center of the widest contiguous run of columns whose count equals the maximum $h_{\max} = \max_x h(x)$; this resolves ties on a plateau of equal counts.

The output is the vertical line $x = x_{\text{target}}$ from y_0 to the bottom of the image (centerline model $m = 0$, $q = x_{\text{target}}$), with confidence $h_{\max}/(H - y_0)$ and the same status thresholds (0.38 and 0.60). The method therefore yields a single horizontal setpoint rather than a line $x(y) = my + q$ and provides no heading estimate. Because the target is the column with the largest count, it can move when the path area changes unevenly across the image.

6.9.3 RANSAC-based contour baseline

The RANSAC-based contour baseline (`pf_ransac.py`) reproduces the classical boundary-fitting methodology prevalent in agricultural vision navigation [1, 14]. Unlike Method 4 and the center-point tracker, it operates directly over the segmented corridor mask without horizontal strip discretization or bottom-up association logic.

Following lower ROI cropping ($y_0 = \lfloor 0.30H \rfloor$) and the same morphological filtering as Method 4, only one connected component is kept among those with at least 500 pixels: the one maximizing its area plus ten times the top row of its bounding box, which favors large components that start lower in the image. For each ROI row containing pixels of this component, the leftmost and rightmost pixels define boundary points $(y, x_L(y))$ and $(y, x_R(y))$, provided the row width $w(y) = x_R(y) - x_L(y) + 1$ satisfies $w(y) \geq 10$ px. This yields two point sets, $\mathcal{P}_L$ and $\mathcal{P}_R$, with at most one point per ROI row; at least ten rows are required.

Boundary lines are estimated independently on $\mathcal{P}_L$ and $\mathcal{P}_R$ using Random Sample Consensus (RANSAC) [15]:

1. Across $T_{\max} = 100$ trials, two points are drawn from the point set by a pseudo-random generator initialized with a fixed seed (`random_state`=42 by default).

2. A candidate linear model $x(y) = m^{(t)}y + q^{(t)}$ is determined from the sampled pair.

3. Inliers are identified as points satisfying $|x_k - (m^{(t)}y_k + q^{(t)})| \leq \tau_{\text{res}}$, with residual threshold $\tau_{\text{res}} = 18.0$ px.

4. The model maximizing inlier cardinality is selected, followed by an ordinary least-squares refit over the complete inlier set.

The result is `lost` if the mean inlier residual exceeds $2\tau_{\text{res}} = 36$ px, if the mean inlier ratio of the two edges is below 0.30, or if the score $0.35\,(\text{inlier ratio}) + 0.30\,(\text{fit score}) + 0.20\,(\text{vertical coverage}) + 0.15\,(\text{width score})$ is below 0.38. Accepted lines are rendered from the highest boundary point to the bottom of the image.

The guidance centerline is calculated as the unweighted average of the left and right boundary models:

$$m_C = \frac{m_L + m_R}{2}, \qquad q_C = \frac{q_L + q_R}{2}. \tag{6.18}$$

By fitting row-wise boundary points directly, this approach avoids strip-tuning parameters. Each trial evaluates all boundary points of one edge (at most 157 for a 224×224 input). With the default fixed seed, repeated runs on the same frame return the same result; run-to-run variation ($\sigma_\omega > 0$) appears only if the seed is changed or left unset.

6.10 Comparative trade-offs and structural limitations

Table 6.3 summarizes the structural differences between the four extractors as implemented. It describes the code, not measured performance; the experimental comparison is reported in Chapter 8.

6.10.1 Structural limitations of Method 4

While Method 4 decouples boundary estimation and avoids the stochastic jitter of RANSAC, it introduces specific structural constraints:

1. **Bottom-up seed dependence:** The tracking process initializes from candidate intervals detected within the bottom six strips ($y \in [H - 6h, H - 1]$).

Table 6.3: Structural comparison of the four geometric extraction algorithms.

Characteristic	`pf_histogram`	`pf_centri`	`pf_ransac`	Method 4
Output reference	Lateral column x_{target}	Line (m_C, q_C)	Line (m_C, q_C)	Line (m_C, q_C)
Orientation (e_θ) estimate	No	Yes	Yes	Yes
Deterministic execution	Yes	Yes	Yes with fixed seed (default)	Yes
Per-frame complexity	$O(W \cdot H)$	$O(W \cdot H)$	$O(W \cdot H + T_{\max} \cdot \lvert\mathcal{P}\rvert)$	$O(W \cdot H)$
Edge decoupling	N/A	No	Yes	Yes
One-sided boundary disturbance affects	Column counts	Fitted center	One edge fit	One edge fit
Bottom-up seed dependence	None	Yes	None	Yes
Local jump threshold	None	Yes	None	Yes
Reference geometry	Vertical line	Straight line	Straight line	Straight line

No minimum support is required for the seed, so initialization fails only if the bottom six strips contain no interval, for example when the mask immediately ahead of the vehicle has large false-negative regions. Furthermore, if the algorithm locks onto a false boundary candidate at the base of the image, the spatial continuity constraint propagates this error upward through subsequent strips. In contrast, `pf_histogram` and `pf_ransac` use all rows of their region of interest at once and do not depend on bottom-up seeding.

2. **Sensitivity to tracking hyperparameters:** The displacement limit $D_4 = \max(28, 0.5\, d_{\text{prev}})$ px and the tolerance of three consecutive strips without a compatible candidate are expressed in pixels of a 224×224 input and depend on resolution and camera mounting. Pitch and roll on uneven terrain shift the corridor in image space; displacements above the limit stop the association early. The frame is then rejected only if fewer than three intervals were accepted or if the later geometric and score checks fail.

3. **Linear corridor model constraint:** Method 4 models both boundaries as straight lines ($x = my + q$). In curved crop rows, contour-following trajectories, or headland turning areas, the linear model introduces systematic approximation error. While algorithms like the multi-ROI formulation of Li et al. [6] adopt cubic B-splines, Method 4 trades curve-fitting capacity for numerical stability and determinism on straight corridors.

4. **Vulnerability to one-sided gaps:** Method 4 associates whole intervals, so both edges are tracked together. Where plants are missing on one side, the path interval in the affected strips can widen or merge with neighboring regions; if the resulting edge displacement exceeds the limit in more than three consecutive strips ($3 \times 6 = 18$ px of image height at $N = 24$), association stops. Every accepted interval supplies both edges, and the frame is rejected if either edge has fewer than two non-clipped samples; the code has no single-edge fallback. `pf_histogram` still outputs a column in these conditions, because it does not track edges.

Chapter 7

Robot Platform and Integration

This chapter describes the hardware and software components of the robot platform used for field validation. It covers the camera configuration, computing hardware, software architecture using ROS, and the deployment of the trained segmentation model for real-time inference.

7.1 Robot hardware and sensors

7.1.1 Robot platform

The experimental platform is a custom-built mobile robot prototype, named Demetra, developed for agricultural inspection and autonomous navigation in row-crop environments. The chassis features a four-wheel steering (4WS) and four-wheel drive (4WD) kinematic configuration. All four wheels are independently driven, and both the front and rear axles provide active steering.

Steering operates in a symmetric contra-steering mode (opposite-phase steering), where the front wheels steer by an angle $\delta_f = \delta$ and the rear wheels steer by an equal and opposite angle $\delta_r = -\delta$. This configuration constrains the instantaneous center of rotation (ICR) to lie on the transverse axis passing through the geometric center of the chassis ($x = 0$). Relative to a conventional front-wheel Ackermann steering model with wheelbase L, symmetric four-wheel contra-steering halves the turning radius from $R = L/\tan\delta$ to:

$$R = \frac{L}{2\tan\delta}. \tag{7.1}$$

This reduction in turning radius enables tight turns at row headlands without requiring the robot to swing wide into adjacent crop beds. In addition, the vehicle

sideslip angle at the center of geometry remains zero under ideal kinematic rolling conditions ($\beta = 0$), so the heading of the chassis aligns with the tangent to the path.

▶ DA COMPLETARE: Dimensioni fisiche e peso del telaio

Inserire i valori geometrici misurati sul prototipo reale:

- Passo L (distanza tra asse anteriore e posteriore in metri) e carreggiata T (distanza trasversale tra le ruote in metri).
- Raggio nominale della ruota r_w e altezza libera da terra del telaio (ground clearance).
- Massa totale del veicolo (kg) comprensiva di pacco batterie, sensori e payload.

The traction subsystem comprises four DJI RoboMaster M3508 brushless DC motors, each coupled to an integrated 19:1 planetary gearbox. Operating at a nominal voltage of 24 V, each motor unit delivers up to 3.0 N·m of continuous torque with a no-load output shaft speed of 482 rpm (peak torque up to 5.0 N·m). Speed regulation is handled by four individual DJI RoboMaster C620 electronic speed controllers (ESCs) implementing field-oriented control (FOC) over a CAN bus interface at 1 Mbps, with continuous current limits of 20 A per channel.

Active steering is actuated by four CubeMars AK70-10 KV100 robotic actuators. Each steering unit integrates a brushless DC motor, a 10:1 planetary gearhead, a 14-bit magnetic absolute encoder, and a motor driver in a single housing. A cross-roller bearing supports simultaneous axial, radial, and tilting loads transmitted from the ground through the wheel uprights. The actuators produce a continuous torque of 8.3 N·m and a peak torque of 24.8 N·m under 24 V operation. The integrated 14-bit encoder provides angular position feedback with a resolution of 0.022°, used for closed-loop position control over a secondary CAN bus network.

Low-level actuator control, safety monitoring, and sensor feedback are executed on a DJI RoboMaster Development Board Type C. The board houses an STM32F407IGH6 microcontroller (ARM Cortex-M4 core running at 168 MHz with hardware floating-point support). Two independent CAN bus controllers interface directly with the four traction ESCs and the four steering actuators. The board also incorporates a 6-axis inertial measurement unit (IMU) mounted on a mechanical vibration-damping structure. Deterministic firmware on the microcontroller executes position and velocity loops at fixed intervals, parses motion commands from the onboard computer, and transmits wheel encoder odometry and IMU telemetry.

7.1.2 Camera configuration

Visual sensing is provided by an Intel RealSense D435 camera mounted on the front section of the robot frame. The sensor connects to the onboard computer



Figure 7.1: Physical mobile robot platform and front camera mounting setup

via a USB 3.2 Gen 1 interface. For the navigation pipeline, the camera operates in RGB streaming mode at a resolution of 640×480 pixels and a frame rate of 30 frames per second (fps). The stream is captured as JPEG-compressed frames to minimize transfer overhead across the ROS 2 transport layer. The camera is positioned at height h above the ground with a downward tilt angle α toward the inter-row path, providing a field of view that encompasses the corridor surface and adjacent crop boundaries in front of the vehicle.

7.1.3 Computing platform

Onboard processing is executed on an NVIDIA Jetson AGX Orin Developer Kit (64 GB version). The computing unit incorporates a 12-core ARM Cortex-A78AE v8.2 64-bit CPU operating at 2.20 GHz and an NVIDIA Ampere architecture GPU with 2048 CUDA cores and 64 Tensor cores. The module delivers up to 275 TOPS of sparse INT8 compute capability and 138 TFLOPS in FP16 Tensor Core operations.

The system shares 64 GB of unified LPDDR5 memory between the CPU and GPU over a 256-bit bus, providing a memory bandwidth of 204.8 GB/s. Persistent storage and dataset logging use a 512 GB NVMe SSD mounted over an M.2 PCIe Gen4 $\times 4$ interface at `/mnt/nvme`, leaving the 64 GB onboard eMMC memory dedicated to the base operating system.

The software stack runs on Ubuntu 22.04.5 LTS with Linux kernel `5.15.148-tegra` and NVIDIA Linux for Tegra (L4T) release R36.4.4 under JetPack 6.1. The complete robotics and deep learning environment is containerized using Docker and the NVIDIA Container Runtime, providing pass-through access to the GPU, hardware accelerators, and USB character devices. Within the container, ROS 2 Humble Hawksbill coordinates image acquisition, neural network inference with ONNX Runtime 1.23.0 using the `CUDAExecutionProvider`, geometric line extraction, and low-level serial communication.

7.2 Software architecture

7.2.1 ROS integration

The software pipeline is implemented using the Robot Operating System (ROS). ROS provides a modular architecture where perception, geometric extraction, and control components communicate through topics and services.

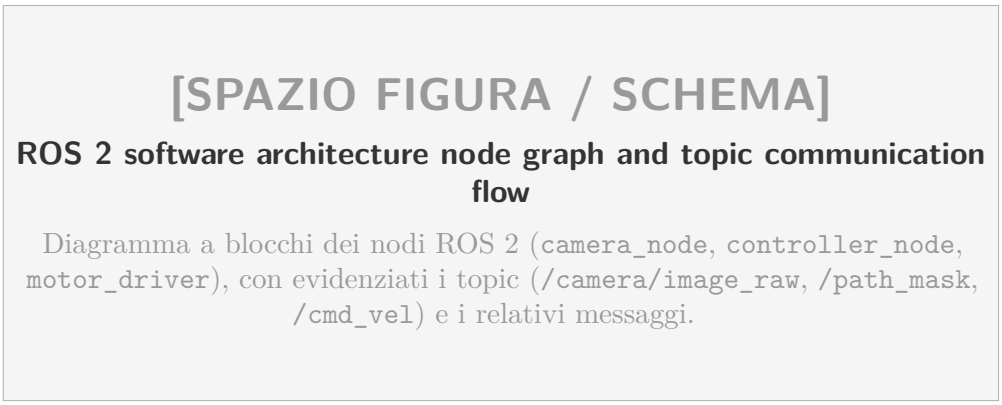


Figure 7.2: ROS 2 software architecture node graph and topic communication flow

▶ **DA COMPLETARE: Architettura dei nodi ROS 2 e politiche QoS**

Descrivere l'organizzazione software nel package ROS 2 `seg_controller_onnx`:

- Grafo dei nodi attivi e rispettivi ruoli nel ciclo di percezione-azione.
- Topic scambiati: ricezione stream telecamera compresso, pubblicazione della maschera del sentiero, coordinate della linea di navigazione e comandi motore `/cmd_vel`.
- Politiche di Quality of Service (QoS): adozione di code a profondità unitaria (`depth=1`) con policy `BEST_EFFORT` per evitare l'accumulo di latenza su frame arretrati.

Perception node

The perception node receives camera images, runs the segmentation model, and publishes the path mask. Input images are preprocessed according to the training configuration before inference.

▶ **DA COMPLETARE: Dettagli implementativi del nodo di percezione**

Descrivere le operazioni eseguite alla ricezione di ogni frame:

- Decodifica dell'immagine compressa JPEG tramite `cv2.imdecode`.
- Ridimensionamento e normalizzazione del tensore d'ingresso (media e deviazione standard di ImageNet).
- Invocazione dell'inferenza tramite la sessione ONNX Runtime con `CUDAExecutionProvider`.
- Estrazione della maschera binaria di navigazione tramite operatore `argmax`.

Geometric extraction node

The geometric extraction node subscribes to the segmentation mask, applies one of the four extractors of Chapter 6 (the proposed Method 4, `pf4.py`, or one of the baselines `pf_centri`, `pf_histogram`, and `pf_ransac`), and publishes the navigation reference as a line or position message. All four extractors are tested on the robot (Chapter 8).

▶ **DA COMPLETARE: Integrazione dell'estratore geometrico e tracking temporale**

Dettagliare l'elaborazione geometrica:

- Suddivisione della maschera in strisce orizzontali e selezione dei candidati.
- Selezione dell'estrattore (`pf4.py`, `pf_centri.py`, `pf_histogram.py` o `pf_ransac.py`) e sua applicazione a ogni frame; indicare dove avviene il filtraggio temporale della linea (ad es. il filtro EMA descritto in questo capitolo).
- Gestione dei frame non validi (reiezione in caso di confidenza o supporto insufficiente).

Control node and error conditioning

The control node converts the image-space geometric reference into linear and angular velocity commands (v, ω) published on the `/cmd_vel` topic. Tracking errors extracted from the segmentation mask at reference row $y_{\text{ref}} = 0.50 \cdot H$ ($y = 112$ px for a 224×224 input resolution) are subject to sensor noise and transient occlusions. To ensure smooth closed-loop actuation without erratic steering adjustments, raw lateral pixel errors e_{px} pass through a multi-stage conditioning filter before entering the steering law.

Dual-regime recursive smoothing and jump rejection. The core filter operates as an exponential moving average (EMA) conditioned on the path finder tracking state $\text{status}_{\text{pf}} \in \{\text{ok}, \text{degraded}\}$. The filter smoothing factor α and jump threshold J_{thr} switch dynamically between two operating regimes:

$$(\alpha, J_{\text{thr}}) = \begin{cases} (0.20,\ 40.0 \text{ px}) & \text{if } \text{status}_{\text{pf}} = \text{ok}, \\ (0.18,\ 38.0 \text{ px}) & \text{if } \text{status}_{\text{pf}} = \text{degraded}. \end{cases} \tag{7.2}$$

At each sampled frame k, the discrepancy between incoming measurement $e_{\text{px},k}$ and prior filtered state e_{k-1} is evaluated as $\Delta_k = |e_{\text{px},k} - e_{k-1}|$. When $\Delta_k \leq J_{\text{thr}}$, the measurement is accepted as nominal and updated via standard EMA:

$$e_k = (1 - \alpha) e_{k-1} + \alpha\, e_{\text{px},k}, \tag{7.3}$$

with the consecutive jump counter reset to zero ($c_{\text{jump}} = 0$).

Conversely, when $\Delta_k > J_{\text{thr}}$, the counter increments ($c_{\text{jump}} \leftarrow c_{\text{jump}} + 1$). If the anomaly occurs on a single isolated frame ($c_{\text{jump}} = 1$), it is treated as a spurious segmentation outlier and heavily attenuated:

$$e_k = 0.80\, e_{k-1} + 0.20\, e_{\text{px},k}. \tag{7.4}$$

If the large displacement persists across consecutive frames ($c_{\text{jump}} \geq 2$), the filter classifies the variation as a genuine path change, resets $c_{\text{jump}} = 0$, and assigns $e_k = e_{\text{px},k}$ directly to avoid phase lag.

Temporal hold and command deadband. When the path finder reports a lost tracking state or when no valid centerline is detected ($e_{\text{px}} = \text{None}$), the filter initiates a zero-order temporal hold:

$$e_k = e_{\text{last_valid}} \quad \text{for } \Delta t \leq t_{\text{hold}}, \tag{7.5}$$

where $t_{\text{hold}} = 0.50$ s (≈ 15 frames at 30 fps). If perception dropouts exceed 0.50 s, the filter clears its internal state and issues an emergency stop command ($v = 0$, $\omega = 0$).

To eliminate high-frequency actuator jitter when the vehicle is aligned with the corridor, an output deadband of ±4.0 px is applied:

$$e_{\mathrm{cmd},k} = \begin{cases} 0.0 & \text{if } |e_k| < 4.0 \text{ px}, \\ e_k & \text{otherwise.} \end{cases} \tag{7.6}$$

The deadband alters only command output $e_{\mathrm{cmd},k}$, preserving internal state e_k to avoid filter deadlock when crossing the zero-error threshold.

Velocity command generation. The filtered lateral error is normalized relative to the image half-width, $e_{\mathrm{norm}} = e_{\mathrm{cmd}}/(W/2)$, and scaled by the geometric extraction confidence $C \in [0, 1]$:

$$\omega = -K_{p,\mathrm{lat}} \cdot (0.6 + 0.4 \cdot C) \cdot e_{\mathrm{norm}}, \tag{7.7}$$

where $K_{p,\mathrm{lat}} = 1.2$ is the lateral proportional gain. Calculated angular velocity is clamped within $[-\omega_{\max}, \omega_{\max}]$, while forward linear speed is set to $v = v_{\mathrm{nom}}$. Table 7.1 summarizes the nominal parameters deployed on the robot platform.

Table 7.1: Nominal parameters of the error conditioning filter and steering controller deployed on Demetra.

Parameter	Symbol	Nominal value
Input resolution ($W \times H$)	–	224×224 px
Reference evaluation row	y_{ref}	112 px ($0.50 \cdot H$)
Smoothing factor (regime `ok`)	α_{ok}	0.20
Smoothing factor (regime `degraded`)	α_{deg}	0.18
Jump threshold (regime `ok`)	$J_{\mathrm{thr,ok}}$	40.0 px
Jump threshold (regime `degraded`)	$J_{\mathrm{thr,deg}}$	38.0 px
Outlier persistence window	N_{persist}	2 frames
Temporal hold duration	t_{hold}	0.50 s ($\approx$ 15 frames)
Output deadband	δ_{db}	±4.0 px
Lateral proportional gain	$K_{p,\mathrm{lat}}$	1.2
Maximum angular velocity	$\omega_{\max}$	0.58 rad/s
Nominal linear speed	v_{nom}	0.20 m/s

7.2.2 Perception pipeline implementation

▶ DA COMPLETARE: Struttura a callback sincrona e carico delle immagini di debug

Descrivere il funzionamento del nodo unificato in un unico thread:

- Sequenzialità del ciclo: decodifica JPEG → inferenza ONNX → estrattore geometrico → generazione comandi → compressione e pubblicazione di 3 immagini di diagnostica visiva.
- Analisi dell'impatto prestazionale: la compressione JPEG delle 3 immagini di debug rappresenta una frazione significativa del tempo totale di callback.

7.2.3 Control interface

The control node translates the guidance reference into velocity commands published as `geometry_msgs/msg/Twist` messages on the `/cmd_vel` topic. These commands specify the forward linear speed v along the longitudinal chassis axis and the angular velocity ω about the vertical yaw axis.

A dedicated serial communication node, `demetra_serial_driver`, interfaces the ROS 2 environment with the low-level STM32 microcontroller. The physical connection uses an FTDI USB-to-UART TTL converter operating at a baud rate of 115200 bps with an update loop running at 100 Hz (10 ms period). Each transmitted packet consists of 24 bytes containing six 32-bit floating-point values followed by a checksum for transmission validation.

On the microcontroller, inverse kinematics converts the target (v, ω) command into individual steering angles and wheel rotation speeds according to the symmetric 4WS model. For a vehicle with wheelbase L and track width T, the nominal steering angle δ is:

$$\delta = \arctan\left(\frac{L\omega}{2v}\right), \tag{7.8}$$

with the front axle steering at $+\delta$ and the rear axle steering at $-\delta$. The individual turn radius for the center of each axle is $R = L/(2\tan\delta)$. To account for track width T and reduce wheel scuffing during cornering, individual steering angles for the left and right wheels follow dual-Ackermann geometry:

$$\delta_L = \arctan\left(\frac{L/2}{R - T/2}\right), \qquad \delta_R = \arctan\left(\frac{L/2}{R + T/2}\right), \tag{7.9}$$

with $\delta_{FL} = \delta_L$, $\delta_{FR} = \delta_R$, $\delta_{RL} = -\delta_L$, and $\delta_{RR} = -\delta_R$. The corresponding wheel linear velocities v_i for each wheel $i \in \{FL, FR, RL, RR\}$ are:

$$v_i = \omega\sqrt{\left(\frac{L}{2}\right)^2 + R_i^2}, \tag{7.10}$$

where R_i is the lateral distance from the ICR to the wheel center plane. Target wheel angular velocities $\omega_{w,i} = v_i/r_w$ (with wheel radius r_w) are converted into motor speed setpoints for the C620 ESCs via the 19:1 gear ratio, while the steering angles are tracked by the AK70-10 actuators via the internal 14-bit encoder position loops.

7.3 Deployment configuration

7.3.1 Model export to ONNX

The deployed LR-ASPP network (the general binary model `best_allsam2.pth`, Section 5.4.4) is exported from PyTorch to ONNX with `torch.onnx.export`. The export script rebuilds the two-class LR-ASPP, loads the checkpoint weights, and wraps the network so that it returns only the main output tensor, that is, the per-pixel class logits; softmax and argmax are not part of the exported graph. The example input is a float32 tensor of size $1 \times 3 \times 224 \times 224$, with the batch dimension declared dynamic and the spatial size fixed at 224×224. The export uses opset 17 with constant folding, and the exported graph is validated with `onnx.checker.check_model`. The ONNX file used on the robot, `lraspp_224x224_allsam2.onnx`, was also compared with the PyTorch checkpoint on the same deterministic random input of size $1 \times 3 \times 224 \times 224$: the maximum absolute difference between the logits was 5.6×10^{-5} and the mean absolute difference 4.9×10^{-6}, so the two outputs agree within a relative tolerance of 10^{-3} and an absolute tolerance of 10^{-4}. This comparison was run with PyTorch 2.6.0 and ONNX Runtime 1.23.2 on the CPU.

On the robot, camera frames are resized to 224×224 before inference, to match this input size. The inference times in Table 5.1 were instead measured at the original image resolution, so they do not describe the onboard model.

7.3.2 Real-time inference

The deployed model processes camera frames at the rate required for closed-loop control. Input preprocessing matches the training transformations. The segmentation mask is decoded and passed to the geometric extraction stage.

▶ DA COMPLETARE: Configurazione del runtime ONNX

Specificare i dettagli dell'ambiente di esecuzione:

- Configurazione dell'allocazione di memoria e provider attivo (`CUDAExecutionProvider`).
- Tipologia di quantizzazione/precisione (es. FP32 standard vs FP16) e consumo di memoria VRAM sulla GPU Jetson.
- Specifiche del tensore d'ingresso (normalizzazione in $[0,1]$ o con media e deviazione standard di ImageNet, risoluzione) e del tensore di uscita (logit o indici di classe, risoluzione).

7.3.3 Timing and latency

End-to-end latency includes image acquisition, preprocessing, segmentation inference, geometric extraction, and control computation. The total pipeline latency must remain below the control update period to maintain stable navigation.

[SPAZIO FIGURA / SCHEMA]

Execution time breakdown of the synchronous perception-control cycle on NVIDIA Jetson

Grafico della scomposizione del tempo di ciclo medio su Jetson: decodifica JPEG (10–15 ms), inferenza ONNX su GPU (10–15 ms), estrazione geometrica (5–10 ms; indicare con quale estrattore è stata misurata), codifica delle 3 immagini JPEG di debug (15–25 ms). Tempo totale per ciclo: 42–76 ms (20 Hz).

Figure 7.3: Execution time breakdown of the synchronous perception-control cycle on NVIDIA Jetson

▶ DA COMPLETARE: Analisi delle latenze misurate e frequenza a 20 Hz (dati reali)

Riportare i risultati empirici ricavati dai test sul Jetson:

- Tempo totale di esecuzione compreso tra 42 e 76 ms per ciclo (media $\approx$ 50 ms), corrispondente a una frequenza di controllo di circa 20 Hz a fronte di una frequenza di cattura della telecamera di 30 fps.
- Analisi del frame dropping indotto dalla callback sincrona con `depth=1` e `BEST_EFFORT` (viene scartato circa un frame ogni tre per mantenere il controllo sul dato più recente).
- Strategie per raggiungere i 30 Hz: (1) pubblicare le immagini di debug a frequenza ridotta o in un thread dedicato; (2) ottimizzare il modello tramite NVIDIA TensorRT; (3) disaccoppiare decodifica e inferenza con architettura multi-thread.

7.3.4 Software-in-the-Loop pre-validation

Before deploying the navigation pipeline on the physical robot, the stability of the full closed-loop chain (segmentation, geometric extraction, error conditioning, steering law, and vehicle kinematics) was verified in a lightweight Software-in-the-Loop (SITL) environment running on a desktop workstation.

SITL architecture

The SITL node loads a single real RGB frame acquired by the Intel RealSense D435 camera on an actual crop row. Rather than rendering a synthetic 3D scene, the simulator models the robot's lateral displacement and heading variation as a horizontal translation of the camera viewport over the original image. The frame is padded with reflective borders (400 pixels per side) to allow displacements of up to ± 0.45 m without introducing black edges.

The pixel shift Δx applied at each simulation step combines two terms derived from the D435 optical parameters:

$$\Delta x = -y \cdot \frac{f_x}{D} + \theta \cdot f_x, \tag{7.11}$$

where y is the lateral offset of the robot from the corridor center (in meters), θ is the heading angle relative to the row axis (in radians), $f_x = 465$ px is the horizontal focal length of the D435 at 640×480 resolution, and $D = 1.25$ m is the lookahead distance corresponding to the reference evaluation row ($y_{\text{ref}} = 0.50 \cdot H$).

Kinematic model

The SITL integrates the same 4WS symmetric contra-steering kinematics implemented in the onboard firmware (`kinematics.cpp`), with identical parameters:

wheelbase $L = 0.40$ m, wheel diameter $d_w = 0.192$ m, and mechanical steering saturation at $\pm 30.0°$. At each time step $\Delta t = 1/30$ s, the node receives (v, ω) from the `/cmd_vel` topic published by the real `controller_node`, computes the effective steering angle:

$$\delta = \arctan\left(\frac{L \cdot \omega}{2\,v}\right), \qquad \omega_{\text{eff}} = \frac{2\,v\,\tan\delta}{L}, \tag{7.12}$$

and updates the planar state:

$$\theta_{k+1} = \theta_k + \omega_{\text{eff}} \cdot \Delta t, \tag{7.13}$$

$$y_{k+1} = y_k - v \sin\theta_k \cdot \Delta t. \tag{7.14}$$

The translated frame is then published on the same ROS 2 topic used by the physical camera (`/camera/camera/color/image_raw/compressed`), so the `controller_node` processes it without any code modification.

Interactive perturbation testing

Keyboard inputs allow the operator to inject controlled lateral perturbations (± 4 cm or ± 10 cm) during execution, simulating sudden displacement events such as wheel slip on soft soil or an external push. This procedure was used to verify that the error conditioning filter and steering controller recover to the corridor center within approximately 10–15 frames (~ 0.5 s at 30 fps), without residual oscillations and without saturating the $\pm 30°$ mechanical steering limit. Table 7.1 lists the filter and control parameters validated through these SITL tests before field deployment.

Chapter 8

Experimental Validation

This chapter reports the experimental evaluation of the segmentation models, geometric extractors, and integrated navigation system. It covers offline evaluation using recorded images and, if completed, closed-loop field trials.

8.1 Offline evaluation protocol

8.1.1 Segmentation metrics

Segmentation performance is measured using intersection over union (IoU) for each semantic class. For a class c, IoU is defined as:

$$\mathrm{IoU}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \tag{8.1}$$

where true positives, false positives, and false negatives are computed against the manual reference masks.

Mean IoU (mIoU) averages the per-class IoU values. For the four-class configuration, mIoU includes background, plants, path, and covering material. Path IoU is the task-specific metric most relevant for navigation.

Architecture comparison results

The architecture comparison in Section 5.3.3 reports, on the validation split of the manual lettuce dataset, a path IoU of 0.9546 for LR-ASPP with a recorded inference time of 10.41 ms, against 0.9621 and 58.24 ms for FCN and 0.9651 and 86.87 ms for DeepLabV3. LR-ASPP accepts an mIoU about 1.2 percentage points lower in exchange for 5.6 to 8.4 times shorter recorded times. Both figures carry the limitations stated in Section 5.3.4: the validation split also selects the checkpoints,

and the time ratios hold only if the three runs were recorded under the same conditions.

Supervision strategy results

[SPAZIO FIGURA / SCHEMA]

Visual comparison of segmentation outputs on lettuce test images and on tomato and cucumber images

Lattuga (immagini di test): immagine RGB, ground truth manuale, predizioni della rete LR-ASPP manuale e delle tre reti SAM 3 addestrate solo sull'insalata. Pomodoro e cetriolo (senza ground truth): immagine RGB e predizione della rete binaria installata sul robot, solo come confronto qualitativo.

Figure 8.1: Visual comparison of segmentation outputs on lettuce test images and on tomato and cucumber images

The supervision comparison is reported in Section 5.4.2. On the test split, the manual model reaches a path IoU of 0.9566 on the complete test set and 0.9615 in the realistic scenario. The three networks trained only on SAM 3 lettuce labels reach path IoU values between 0.8966 and 0.9013 in the realistic scenario and between 0.3917 and 0.4124 on the complete test set; the 4-class network reaches an mIoU of 0.5925 in the realistic scenario. A network with the deployed general configuration, trained on the complete SAM 3 collection without the lettuce test images, reaches 0.9071 in the realistic scenario and 0.6369 on the complete test set; the deployed weights themselves have no independent IoU (Section 5.4.4).

8.1.2 Geometric reference accuracy

Geometric reference quality is assessed by evaluating four extraction strategies on identical segmentation masks: the proposed dual-edge extractor with continuity tracking (Method 4), the RANSAC-based contour baseline (`pf_ransac.py`), the center-point strip tracking baseline (`pf_centri.py`), and the column-histogram baseline (`pf_histogram.py`). Performance metrics comprise:

- Reference line parameters: estimated slope m_C and lateral intercept q_C (for linear models);

- Lateral deviation e_y and heading orientation error e_θ evaluated against manual centerline annotations;
- Sensitivity to asymmetric disturbances, quantified by the lateral deviation induced by unilateral crop intrusions or shadows;
- Rejection rate: percentage of test images where extraction fails validation checks (status classified as lost);
- Execution time per frame measured on the onboard NVIDIA Jetson Orin Nano platform.

[SPAZIO FIGURA / SCHEMA]

Comparative evaluation of geometric reference extraction across the four evaluated algorithms

Confronto qualitativo dell'estrazione della linea di guida su maschere di test in tre scenari: (a) filare uniforme; (b) filare con buchi/interruzioni di piante; (c) filare con ombre e intrusioni fogliari asimmetriche. Mostrare le linee stimate da Metodo 4, pf_ransac, pf_centri e il target dell'istogramma a colonne pf_histogram.

Figure 8.2: Comparative evaluation of geometric reference extraction across the four evaluated algorithms

Quantitative comparison of geometric extractors

Table 8.1 reports offline performance metrics for the four geometric extraction algorithms evaluated on recorded test sequences. Each algorithm processes identical segmentation masks predicted by the deployed binary network (`best_allsam2.pth`, Section 5.4.4) on the lettuce, tomato, and cucumber datasets.

Angular jitter σ_ω measures the standard deviation of the estimated centerline slope across 100 repeated executions on the same input frame. Deterministic algorithms produce $\sigma_\omega = 0.00$ by definition. `pf_ransac` draws its samples from a generator with a fixed default seed, so it is also reproducible on the same frame; it shows non-zero jitter only if the seed changes between executions, which must be stated with its result.

Table 8.1: Offline benchmark of geometric extraction algorithms on multi-crop test sequences.

Metric	`pf_histogram`	`pf_centri`	`pf_ransac`	Method 4
Lateral RMSE (cm)	—	—	—	—
Angular RMSE (deg)	—	—	—	—
Angular jitter σ_ω (deg)	0.00	0.00	—	0.00
Rejection rate (%)	—	—	—	—
Execution time (ms)	—	—	—	—
Deterministic	Yes	Yes	Yes (fixed seed)	Yes

Asymmetric disturbance sensitivity

Asymmetric corridor disturbances—such as protruding leaves, localized weeds, or shadows extending along one side—test the ability of each algorithm to maintain corridor centering when the segmentation mask exhibits unilateral bias. Algorithms that collapse bilateral boundaries into a single centerline estimate before fitting (such as `pf_centri`) shift laterally toward the intrusion. Method 4 and `pf_ransac` isolate each boundary and compute the centerline analytically, reducing sensitivity to unilateral noise.

Failure mode analysis

Extraction failures occur when validation checks reject the computed reference as unreliable. Common failure modes include:

- **Tracking loss:** Insufficient mask continuity across horizontal strips prevents bottom-up tracking initialization.
- **Angular drift from corrupted centroids:** Single outlier intervals dominate the centroid-based fit in `pf_centri`, producing reference lines with excessive slope.
- **Histogram target jumps:** The column-histogram minimum in `pf_histogram` shifts abruptly between frames when corridor width varies or when noise perturbs column sums.
- **RANSAC convergence variability:** In sparse or highly noisy masks, `pf_ransac` may fail to identify a consistent inlier set within the iteration budget, leading to sporadic rejections.

8.2 Field validation

Field validation evaluates the integrated navigation system on the physical robot Demetra operating autonomously inside crop rows. Unlike offline evaluation, which isolates the geometric extractor on recorded images, closed-loop field tests exercise the full chain from camera acquisition through segmentation, geometric fitting, error conditioning, and steering actuation in the presence of real-world disturbances.

8.2.1 Ground-truth reference: laser sensor

To assess the robot's physical centering independently of the onboard vision pipeline under test, closed-loop field validation integrates an auxiliary laser sensor.

▶ **DA COMPLETARE: Dettagli del sensore laser utilizzato per i test in campo**

Inserire le specifiche esatte del sensore laser impiegato nei test:

- Modello, costruttore e tipologia di misura (ad es. sensore di distanza laser, barriera ottica o profilometro).
- Configurazione di montaggio sul robot Demetra (posizione e orientamento).
- Interfaccia di comunicazione e frequenza di campionamento.

The laser measurement provides an independent reference signal to monitor the robot's lateral alignment along the crop row and evaluate whether the trajectory remains within the allowable tolerance corridor.

8.2.2 Field evaluation metrics

The following metrics are computed for each test run. All algorithms under comparison (Method 4, `pf_centri`, `pf_ransac`, and `pf_histogram`) are evaluated on the same physical row under equivalent conditions.

Laser centering rate

The laser centering rate P_{laser} quantifies the fraction of the autonomous run during which the robot remains centered within the laser reference corridor:

$$P_{\text{laser}} = \frac{T_{\text{on}}}{T_{\text{total}}} \times 100 \ [\%], \tag{8.2}$$

where T_{on} is the cumulative time with positive laser detection and T_{total} is the total duration of the autonomous run. A higher P_{laser} indicates that the guidance algorithm maintains the robot closer to the physical corridor center for a larger fraction of the mission.

Number of centering losses

The number of centering loss events N_{exit} counts how many times the laser detector transitions from active (1) to inactive (0) during a run. Each event corresponds to the robot drifting outside the acceptable centering band. Frequent transitions indicate oscillatory tracking behavior, even if the overall centering rate P_{laser} remains acceptable.

Mission completion rate

The mission completion rate S_{mission} measures the fraction of the test row traversed autonomously without requiring manual intervention or emergency stop:

$$S_{\text{mission}} = \frac{L_{\text{auto}}}{L_{\text{row}}} \times 100\ [\%], \tag{8.3}$$

where L_{row} is the total length of the test row and L_{auto} is the distance covered before any intervention. A run terminated by collision with crop plants or by operator emergency stop records $S_{\text{mission}} < 100\%$.

Steering stability

Steering stability is evaluated from the angular velocity commands ω_k recorded on the `/cmd_vel` topic. Two indicators are computed:

- The standard deviation of angular velocity, $\sigma_\omega = \text{std}(\omega_k)$, which quantifies overall steering activity;
- The mean absolute jitter, $J_\omega = \frac{1}{N-1} \sum_{k=1}^{N-1} |\omega_{k+1} - \omega_k|$, which isolates high-frequency direction reversals independent of steady-state turning.

Lower values of both indicators correspond to smoother robot motion and reduced mechanical wear on the CubeMars AK70-10 steering actuators.

Effective forward speed

The effective forward speed v_{eff} is the ratio between the distance traversed and the elapsed time. If the error conditioning filter enters the temporal hold state and issues stop commands ($v = 0$), the effective speed drops below the nominal setpoint $v_{\text{nom}} = 0.20$ m/s. This metric captures how frequently each algorithm loses the guidance reference and forces the robot to halt.

8.2.3 Test conditions

▶ DA COMPLETARE: Descrizione delle condizioni sperimentali in campo

Dettagliare lo scenario delle prove sperimentali:

- Località del test, tipologia di coltura, sesto d'impianto (distanza tra i filari e altezza delle piante).
- Lunghezza del percorso navigato, tipologia del fondo (terreno lavorato, teli plastici pacciamanti, presenza di malerbe).
- Configurazione del sensore laser di riferimento (modello, posizionamento sul robot o nel filare, principio di misura).
- Condizioni meteorologiche e di luce (sole diretto, cielo coperto, ombre proiettate dalla vegetazione) e numero di ripetizioni effettuate per ciascun algoritmo.

8.2.4 Closed-loop navigation results

[SPAZIO FIGURA / SCHEMA]

Lateral tracking error and heading error profiles during inter-row autonomous navigation

Grafico dell'errore di inseguimento laterale e_y (cm) e dell'errore di orientamento e_θ (gradi) registrati nel tempo durante la navigazione del robot mobile all'interno dei filari coltivati.

Figure 8.3: Lateral tracking error and heading error profiles during inter-row autonomous navigation

▶ DA COMPLETARE: Risultati quantitativi di navigazione ad anello chiuso

Riportare i dati raccolti durante le prove in campo con il robot Demetra:

- Tabella comparativa tra i 4 algoritmi (Metodo 4, pf_ransac, pf_centri, pf_histogram) con le metriche definite in Sezione 8.2.2: P_{laser}, N_{exit}, S_{mission}, σ_ω, J_ω e v_{eff}.
- Grafico temporale del segnale del sensore laser sovrapposto alla velocità angolare comandata $\omega(t)$ per ciascun algoritmo.
- Analisi degli eventi critici: corrispondenza tra le anomalie del segnale laser e i frame classificati come `lost` dal path finder.

8.2.5 Qualitative observations

▶ **DA COMPLETARE: Osservazioni qualitative sul comportamento del sistema**

Discutere gli aspetti qualitativi emersi durante i test:

- Differenze visibili nella fluidità di guida tra Metodo 4 e pf_centri (oscillazioni laterali, scatti di sterzo).
- Comportamento in presenza di interruzioni (gaps) nella fila di piante e al variare dell'illuminazione.
- Casi limite riscontrati (es. testate del campo, biforcazioni del corridoio, erbacce invadenti).

8.3 Discussion

8.3.1 Segmentation performance

The architecture comparison supports LR-ASPP as an accuracy–speed compromise: on the manual lettuce validation split it loses about 1.2 percentage points of mIoU with respect to FCN and DeepLabV3, with much shorter recorded inference times. IoU alone does not establish that a segmentation is sufficient for navigation; this is assessed through the geometric extraction and the robot tests.

The supervision comparison tested labels generated by SAM 3 and used without any manual correction. On lettuce, networks trained on these labels remain below the manual model (path IoU 0.9013 against 0.9615 for the binary network in the realistic scenario) and lose most of their accuracy on the complete test set. Part of this gap may come from a difference in label definition: the manual masks mark only the central traversable corridor as path, whereas networks trained on SAM 3 labels can also segment adjacent or non-traversable corridors, which the IoU counts as false positives. This behavior was observed visually for the deployed network but was not quantified. The deployed network required no manual annotation of its training data; the time saved with respect to manual labeling was not measured.

8.3.2 Geometric extraction trade-offs

The four geometric extraction methods involve different trade-offs, and none is best on every criterion:

- **Method 4 versus `pf_ransac`:** Method 4 is deterministic and needs no random hypothesis generation; `pf_ransac` is reproducible only through its fixed seed and evaluates all boundary points in each of its 100 trials. `pf_ransac` operates on row-wise boundary points without horizontal discretization or

sequential gating. When canopy boundaries contain extended irregular gaps, RANSAC can still find inliers above and below the gap, whereas Method 4 stops the association after more than three consecutive strips without a compatible candidate.

- **Method 4 versus `pf_centri`:** By maintaining separate linear models for the left and right corridor boundaries, Method 4 isolates asymmetric disturbances such as localized weed patches or protruding leaves. In contrast, `pf_centri` collapses interval endpoints into midpoints prior to fitting, causing unilateral border noise to directly perturb the estimated centerline. Nevertheless, `pf_centri` requires fewer validation checks and operates with a simpler association logic than the decoupled dual-edge architecture.

- **Method 4 versus `pf_histogram`:** The column-histogram baseline (`pf_histogram`) is computationally the simplest approach, computing column projections in $O(W \cdot H)$ without strip partitioning, boundary association, or matrix inversion. It does not suffer from tracking divergence or initialization failure. Its primary drawback is geometric incompleteness: it produces only a horizontal setpoint x_{target} without estimating heading error e_θ. Furthermore, its target shifts when vegetation area varies unevenly along the corridor. Method 4 delivers a complete linear model (m_C, q_C) supporting full pose feedback, but incurs higher algorithmic complexity and depends on bottom-up mask continuity.

8.3.3 Limitations

The experimental findings and the proposed methodology are subject to several specific limitations:

- **Seed dependency in Method 4:** The bottom-up tracking logic relies on detecting a valid corridor interval within the bottom six strips of the region of interest. If the mask immediately in front of the vehicle is corrupted by motion blur, ground glare, or chassis shadows, tracking fails to initialize, causing frame rejection even when the upper corridor is clearly segmented. Unlike global projection methods, Method 4 lacks a mechanism to re-anchor tracking from the top down.

- **Fixed spatial gate calibration:** The continuity threshold $\Delta x_{\max} = 28$ px and strip discretization parameters are fixed in image coordinates. Large vehicle pitch or roll on rough terrain causes rapid lateral shifts of the corridor boundaries between consecutive frames. When these shifts exceed the gate, the tracker rejects valid boundaries.

- **Straight-line corridor assumption:** All linear models evaluated (Method 4, `pf_ransac`, and `pf_centri`) assume locally straight, parallel corridor boundaries. They cannot follow curved planting geometries or guide the robot through headland turnaround maneuvers where corridor boundaries terminate or diverge.
- **Segmentation evaluation scope:** Quantitative segmentation evaluation is limited to the lettuce test images. The deployed weights have no independent IoU, because their training data include the lettuce test images; the general configuration is evaluated through a separate network trained without those images, and tomato and cucumber have no manual ground truth. The deployed network was also assessed by visual inspection of its predictions, in simulation, and in field tests. In addition, the random image-level split takes training and test images from the same lettuce folders, each recorded along one path; no test image is used for training, but the reported IoU values may overestimate performance on paths or fields not seen during training.
- **Offline versus closed-loop dynamics:** Offline benchmarks quantify reference line accuracy on static recorded frames. They do not account for dynamic wheel-ground interaction, actuator saturation, or latency jitter during physical vehicle operation.

Chapter 9

Conclusions and Future Work

This chapter summarizes the main findings, discusses the contributions and limitations of the work, and identifies directions for future research.

9.1 Main contributions

This thesis investigated vision-based local guidance for agricultural row navigation through segmentation model selection, dataset preparation with automatic SAM 3 annotation, and geometric reference extraction. The main contributions are:

- **Architecture comparison for robotic segmentation:** FCN, DeepLabV3, and LR-ASPP were compared on manually annotated lettuce images. LR-ASPP was selected for its accuracy–speed compromise: on the validation split it reached a path IoU of 0.9546 with a recorded inference time of 10.41 ms, against 58.24 ms and 86.87 ms for the other architectures, at the cost of an mIoU about 1.2 percentage points lower.
- **Automatic dataset annotation with SAM 3:** Training labels were generated with SAM 3 text prompts and used without any manual correction. Networks trained on these labels for lettuce were compared with a network trained on manual masks, and the binary labeling was then extended to the complete lettuce, tomato, and cucumber collection to train the network deployed on the robot.
- **Evaluation of output label configurations:** Networks with four-class, three-class, and binary path outputs, trained on SAM 3 lettuce labels, were compared against the manual ground truth. Their path IoU values differ by less

than 0.5 percentage points in the realistic scenario. The binary configuration was adopted for the robot because the path is the only class used by the geometric extraction.

- **Edge-based geometric extractor:** An extractor that converts corridor masks into image-space navigation references was implemented (`pf4.py`, Method 4). It uses horizontal strip extraction, bottom-up interval association, and independent fitting of the left and right boundaries, and it is compared with three baseline algorithms (`pf_centri`, `pf_histogram`, `pf_ransac`).

9.2 Conclusions on segmentation and annotation

The architecture comparison supports LR-ASPP as a compromise between segmentation accuracy and recorded inference time on the robot hardware. Whether its segmentation is sufficient for navigation is assessed through the geometric extraction and the robot tests, not from IoU alone.

On lettuce, the manual model remains the best: path IoU 0.9566 on the complete test set and 0.9615 in the realistic scenario. Networks trained only on uncorrected SAM 3 labels reach a path IoU of about 0.90 in the realistic scenario (0.9013 for the binary network), but only 0.39–0.41 on the complete test set, and the 4-class network reaches an mIoU of 0.5925. Fully automatic labels therefore provide path supervision of about 0.90 IoU in the operational conditions, but they are not a replacement for manual annotation when all classes are needed or when the scene departs from those conditions. The general binary configuration used on the robot, trained on the complete SAM 3 collection without any manual annotation, reaches 0.9071 in the realistic scenario and 0.6369 on the complete test set when the lettuce test images are excluded from its training data. The deployed weights were assessed visually, in simulation, and on the robot.

The choice of output label configuration affects both annotation effort and downstream geometric extraction. Binary path segmentation focuses the model directly on the navigation target, simplifying both the labeling task and the geometric stage. This comes at the cost of discarding plant and covering-material information that might be useful for other tasks.

9.3 Conclusions on geometric extraction

The geometric extraction stage converts binary corridor segmentation masks into navigation setpoints. The comparative study examined four distinct architectural paradigms:

1. Global column projection (`pf_histogram`) maximizes simplicity and eliminates tracking failure modes, but discards heading angle information and exhibits sensitivity to asymmetric vegetation distribution.

2. Centerline strip regression (`pf_centri`) provides both lateral offset and heading estimates with low computational demand, but transfers unilateral canopy noise directly into the navigation line.

3. Random sample consensus (`pf_ransac`) handles scattered boundary points and wide gaps without strip discretization, but relies on random sampling: its result is reproducible only with a fixed seed, and each trial evaluates all boundary points.

4. Decoupled dual-edge tracking (Method 4) provides deterministic execution and boundary-level noise decoupling, but introduces strict dependencies on near-field seed detection and heuristic jump gates.

These characteristics demonstrate that no individual extractor is universally superior; selection depends on sensor setup, platform compute limits, and field layout.

▶ DA COMPLETARE: Sintesi quantitativa finale dell'estrazione geometrica e navigazione

Sintetizzare i dati conclusivi:

- Margine di precisione medio dell'asse stimato (scostamento in pixel e in centimetri).
- Tasso medio di affidabilità e di reiezione dei frame lungo i test eseguiti.
- Conferma della sostenibilità del controllo a 20 Hz su piattaforma embedded Jetson in condizioni di campo.

9.4 Limitations

The methodology and experimental findings are subject to several specific limitations:

- **Bottom-up seed dependency in Method 4:** The tracking algorithm initializes exclusively from the bottom six horizontal strips. Occlusions immediately in front of the camera (such as robot chassis shadows, soil buildup, or puddles) prevent track initialization and lead to frame rejections, even when the distant corridor is visible.

- **Geometric model constraints:** Method 4, `pf_ransac`, and `pf_centri` assume locally straight, parallel corridor boundaries. They cannot follow

curved crop rows or navigate headland turnaround maneuvers where crop boundaries terminate or diverge.

- **One-sided gap vulnerability:** Method 4 tracks whole intervals, so both edges are associated together. If missing plants on one side make the edge displacement exceed the limit in more than three consecutive strips (18 px), association stops, and the frame is rejected if either edge has fewer than two non-clipped samples. There is no single-edge fallback, whereas the column histogram still provides a lateral setpoint.

- **Image-space gate tuning:** The continuity gate ($\Delta x_{\max} = 28$ px) is defined in image coordinates. Large vehicle pitch and roll on uneven soil cause sudden lateral displacements of the corridor between consecutive scanlines, which can trigger premature tracking loss.

- **Dataset annotation coverage:** Segmentation accuracy was measured against manual ground truth only on lettuce. The deployed weights have no independent IoU, because their training data include the lettuce test images; the general configuration was evaluated through a separate network trained without them. Tomato and cucumber have no manual ground truth. The random image-level split also takes training and test images from the same lettuce folders, each recorded along one path, so the reported IoU values may be optimistic for paths not seen during training.

- **Label definition mismatch:** The manual masks mark only the central traversable corridor as path, whereas networks trained on SAM 3 labels can also segment adjacent or non-traversable corridors. This lowers their IoU against the manual masks, but the size of the effect was not measured.

- **Quality of the SAM 3 labels:** Visual inspection showed that SAM 3 segmented the plastic mulch film poorly and also assigned the greenhouse plastic cover to the covering class, and that its path masks were less accurate in images taken from unusual viewpoints or where the path between the rows is not clearly visible. Because the labels were used without correction, these errors enter the training data; their extent was not measured.

- **Annotation effort measurement:** The time saved by automatic labeling with respect to manual annotation was not measured.

- **Closed-loop dynamics:** Offline benchmarks evaluate reference accuracy on static test images. They do not account for vehicle inertia, steering backlash, or dynamic latency variations under closed-loop field control.

9.5 Future work

Several directions for future research emerge from this work:

9.5.1 Dataset and annotation

- Measure the annotation time of manual and automatic workflows, and test whether a light correction of the SAM 3 labels improves the trained networks.
- Split the data by folder rather than by image, so that the test paths are not represented in training, and repeat the evaluation of all networks.
- Prepare manual ground-truth masks for tomato and cucumber to evaluate the deployed network on those crops directly.
- Investigate active learning strategies to select images for manual annotation that improve model generalization most effectively.
- Explore other foundation models or fine-tuned SAM variants specialized for agricultural segmentation.

9.5.2 Segmentation and model deployment

- Evaluate additional lightweight architectures (e.g., BiSeNet, FastSCNN) for further latency reduction.
- Investigate quantization and model compression techniques to enable deployment on edge devices.
- Test temporal consistency methods that use previous frames to improve segmentation and reduce flickering.
- Explore uncertainty estimation to detect when segmentation confidence is low and fallback strategies are needed.

9.5.3 Geometric extraction

- Extend the geometric representation from linear models to piecewise linear or polynomial formulations to support curved rows and headland turns.
- Implement top-down recovery logic and single-edge asymmetric tracking to maintain guidance when bottom-up seeding fails or unilateral row dropouts occur.

- Adapt spatial gate thresholds dynamically using vehicle attitude telemetry from an onboard IMU to prevent false tracking loss during pitch and roll variations.
- Incorporate temporal Bayesian tracking (such as an Extended Kalman Filter) to smooth reference parameters across frame dropouts.

9.5.4 System integration and field validation

- Complete closed-loop field trials to validate end-to-end navigation performance under variable lighting, growth stages, and row conditions.
- Measure lateral tracking error and success rate over extended distances and multiple crop types.
- Integrate temporal filtering and prediction to handle brief reference loss or occlusions.
- Investigate fusion with other sensors (e.g., depth, IMU) to keep a valid reference when the camera view is degraded.
- Develop a complete autonomy system including global path planning, obstacle avoidance, and task-specific behaviors.

9.6 Closing remarks

This thesis addressed the segmentation, dataset preparation, and geometric extraction stages of vision-based agricultural navigation. The selection of LR-ASPP balances accuracy and speed for robotic deployment. Labels generated by SAM 3 without manual correction were used to train the network deployed on the robot, and the geometric extractor converts masks into navigation references using horizontal strip selection and independent edge fitting.

The work establishes a foundation for camera-based local guidance in structured crop environments. Future research should complete field validation, extend the geometric model to handle curves and gaps, and evaluate the deployed network on independent, sequence-level test data.

Bibliography

[1] Yuhao Bai, Baohua Zhang, Naimin Xu, Jun Zhou, Jiayou Shi, and Zhihua Diao. «Vision-based navigation and guidance for agricultural autonomous vehicles and robots: A review». In: *Computers and Electronics in Agriculture* 205, 107584 (Feb. 2023). DOI: 10.1016/j.compag.2022.107584 (cit. on pp. 1, 2, 4, 5, 9, 13, 62).

[2] Rajitha de Silva, Grzegorz Cielniak, Gang Wang, and Junfeng Gao. «Deep Learning-Based Crop Row Detection for Infield Navigation of Agri-Robots». In: *Journal of Field Robotics* 41.7 (2024), pp. 2299–2321. DOI: 10.1002/rob.22238 (cit. on pp. 1, 4, 7–10, 13, 15, 16).

[3] Alessandro Navone, Mauro Martini, Marco Ambrosio, Andrea Ostuni, Simone Angarano, and Marcello Chiaberge. «GPS-Free Autonomous Navigation in Cluttered Tree Rows with Deep Semantic Segmentation». In: *Robotics and Autonomous Systems* 183, 104854 (Jan. 2025). DOI: 10.1016/j.robot.2024.104854 (cit. on pp. 1, 2, 4, 7, 8, 10–12, 15, 16, 62).

[4] Enrico Mendez, Javier Piña Camacho, Jesús Arturo Escobedo Cabello, and Alfonso Gómez-Espinosa. «Autonomous Navigation and Crop Row Detection in Vineyards Using Machine Vision with 2D Camera». In: *Automation* 4.4 (Sept. 2023), pp. 309–326. DOI: 10.3390/automation4040018 (cit. on pp. 2, 5, 7–9, 12, 15, 16).

[5] Johannes Kneip, Patrick Fleischmann, and Karsten Berns. «Crop Edge Detection Based on Stereo Vision». In: *Robotics and Autonomous Systems* 123, 103323 (Jan. 2020). DOI: 10.1016/j.robot.2019.103323 (cit. on pp. 4, 7, 12, 14–16).

[6] Xia Li, Junhao Su, Zhenchao Yue, and Fangtao Duan. «Adaptive Multi-ROI Agricultural Robot Navigation Line Extraction Based on Image Semantic Segmentation». In: *Sensors* 22.20, 7707 (Oct. 2022). DOI: 10.3390/s22207707 (cit. on pp. 5, 7, 10, 12, 14, 15, 17, 65).

[7] Jiqing Chen, Hu Qiang, Jiahua Wu, Guanwen Xu, and Zhikui Wang. «Navigation Path Extraction for Greenhouse Cucumber-Picking Robots Using the Prediction-Point Hough Transform». In: *Computers and Electronics in Agriculture* 180, 105911 (Jan. 2021). DOI: 10.1016/j.compag.2020.105911 (cit. on pp. 5, 7, 12, 15, 16).

[8] Alessandro Navone, Mauro Martini, Andrea Ostuni, Simone Angarano, and Marcello Chiaberge. «Autonomous Navigation in Rows of Trees and High Crops with Deep Semantic Segmentation». In: *2023 European Conference on Mobile Robots (ECMR)*. IEEE, 2023, pp. 1–6. DOI: 10.1109/ECMR59166.2023.10256334 (cit. on p. 10).

[9] Jonathan Long, Evan Shelhamer, and Trevor Darrell. «Fully Convolutional Networks for Semantic Segmentation». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2015, pp. 3431–3440. DOI: 10.1109/CVPR.2015.7298965 (cit. on pp. 10, 18–22).

[10] Liang-Chieh Chen, George Papandreou, Florian Schroff, and Hartwig Adam. «Rethinking Atrous Convolution for Semantic Image Segmentation». In: *arXiv preprint arXiv:1706.05587* (2017) (cit. on pp. 10, 20, 22–24).

[11] Andrew Howard et al. «Searching for MobileNetV3». In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019, pp. 1314–1324 (cit. on pp. 10, 24–27, 29).

[12] Alexander Kirillov et al. «Segment Anything». In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. Oct. 2023, pp. 4015–4026. arXiv: 2304.02643 [cs.CV] (cit. on pp. 11, 30, 31).

[13] Nicolas Carion et al. «SAM 3: Segment Anything with Concepts». In: *International Conference on Learning Representations (ICLR)*. 2026. arXiv: 2511.16719 [cs.CV]. URL: https://proceedings.iclr.cc/paper_files/paper/2026/hash/e0982cbc81401df3430ee1ff780dc7a2-Abstract-Conference.html (cit. on pp. 11, 30, 31).

[14] Letian Wu, Yongzhi Cui, Huifeng Shi, Xiaoli Sun, Jiayan Yang, Xinwei Cao, Ping Zou, and Ya Liu. «An Improved DeepLabV3+-Based Method for Crop Row Segmentation and Navigation Line Extraction in Agricultural Fields». In: *Sensors* 26.10, 3142 (May 2026). DOI: 10.3390/s26103142 (cit. on pp. 13, 62).

[15] Martin A. Fischler and Robert C. Bolles. «Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography». In: *Communications of the ACM* 24.6 (June 1981), pp. 381–395. DOI: 10.1145/358669.358692 (cit. on pp. 13, 62).

[16] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn and PyTorch: Concepts, Tools, and Techniques to Build Intelligent Systems*. 1st ed. O'Reilly Media, 2026. ISBN: 979-8-341-60798-9 (cit. on pp. 18–21, 23, 39).

[17] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. «Deep Residual Learning for Image Recognition». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90 (cit. on p. 20).

[18] TorchVision Maintainers and Contributors. *TorchVision: Models and Pretrained Weights — FCN-ResNet50*. URL: https://pytorch.org/vision/stable/models/fcn.html (visited on 10/23/2026) (cit. on p. 22).

[19] TorchVision Maintainers and Contributors. *TorchVision: DeepLabV3 segmentation models, source code* ***torchvision/models/segmentation/deeplabu3.py***. URL: https://github.com/pytorch/vision/blob/main/torchvision/models/segmentation/deeplabv3.py (visited on 10/23/2026) (cit. on p. 24).

[20] TorchVision Maintainers and Contributors. *TorchVision: LR-ASPP segmentation model, source code* ***torchvision/models/segmentation/lraspp.py***. URL: https://github.com/pytorch/vision/blob/main/torchvision/models/segmentation/lraspp.py (visited on 10/23/2026) (cit. on p. 27).

[21] NumPy Developers. *numpy.polyfit: NumPy v2.4 Manual*. URL: https://numpy.org/doc/2.4/reference/generated/numpy.polyfit.html (visited on 09/13/2026) (cit. on p. 55).